Security Fundamentals

A Practical Guide

- Understand what cybersecurity is, from the ground up
- Learn how attacks work, and how to defend
- Build the base for every later security course





Angkor Wat, Angkor
UNESCO World Heritage · inscribed 14 December 1992

TOUCH DEN

Contents

Section 1: What Cybersecurity Is 1

1. A Real Incident: How One Email Locked a Business 2
2. What Cybersecurity Is 3
3. Why It Matters Now 3
4. What We Protect: Information and Systems 4
5. The Three Questions 5
6. Threats, Vulnerabilities, and Risk 5
7. Defense Is Layers 6
8. Security Is Everyone's Job 6
9. Why We Learn to Attack 7
10. Ethics and the Law: The One Rule 7
11. The Road Ahead 8

Section 2: The CIA Triad and Beyond 12

1. Why We Need Clear Goals 13
2. Confidentiality 13
3. Integrity 14
4. Availability 15
5. Authentication, Authorization, and Accounting (AAA) 15
6. Non-Repudiation 16
7. The Trade-Offs Between Them 16
8. Applying the Goals 17

Section 3: Threats, Vulnerabilities, and Risk 20

1. The Three Words, Precisely 21
2. Where Vulnerabilities Come From 22
3. Threat Sources and Threat Events 22
4. Likelihood and Impact 23
5. Rating Risk: Qualitative and Quantitative 23
6. The Four Ways to Treat a Risk 24
7. Residual Risk 24
8. A Simple Risk Register 25

Section 4: Who Attacks and Why 29

1. The Myth of the Hooded Hacker 30
2. Script Kiddies and Opportunists 31
3. Cybercriminals and Ransomware Crews 31
4. Insiders 32
5. Hacktivists 32
6. Nation-State and Advanced Persistent Threats 32
7. Motivations: Money, Information, Disruption, and Belief 33
8. What This Means for Defending 34

Section 5: Common Attacks: Phishing, Malware, and More 37

1. How Most Attacks Actually Start 38
2. Phishing and Social Engineering 39
3. Malware: Viruses, Worms, Trojans, and Ransomware 40
4. Password Attacks 40
5. Attacks on Web and Network Services 41
6. Denial of Service 41
7. Man-in-the-Middle 42
8. The Attack Lifecycle 42

Section 6: Passwords and Authentication 46

1. What Authentication Is 47
2. The Three Factors 48

3. Why Passwords Fail 48

4. How Passwords Are Stored and Cracked 49

5. Making Strong Passwords and Passphrases 49

6. Password Managers 50

7. Multi-Factor Authentication 50

8. Passkeys: The Direction of Travel 51

Section 7: Access Control Basics 55

1. Authentication vs. Authorization 56

2. Least Privilege 56

3. Separation of Duties 57

4. Access Control Models 58

5. Users, Groups, and Roles 58

6. Privilege Creep 59

7. Access Control in Practice 59

Section 8: Security Controls and Defense in Depth 63

1. What a Control Is 64

2. Control Types: Preventive, Detective, Corrective 65

3. Control Categories: Technical, Administrative, Physical 65

4. Defense in Depth, Properly 66

5. Layered Defense: A Worked Example 67

6. Where Controls Fail 67

7. Security vs. Usability 68

Section 9: Ethics, the Law, and Permission to Test 71

1. Why Ethics Comes First 72

2. The One Rule, Restated 72

3. Authorization, Scope, and Rules of Engagement 73

4. Bug Bounties and Responsible Disclosure 74

5. The Shape of Computer-Crime Law 74

6. Data Protection and Privacy Duties 75

7. Professional Ethics and What to Do If You Find Something 75

Section 10: Security Frameworks and Standards 79

1. Why Frameworks Exist 80

2. The NIST Cybersecurity Framework 2.0 81

3. The CIS Critical Security Controls 81

4. ISO/IEC 27001 and 27002 82

5. MITRE ATT&CK 82

6. Compliance vs. Security 83

7. Choosing Where to Start 83

Section 11: Security Roles and the Industry 87

1. The Shape of the Industry 88

2. Offensive Roles 89

3. Defensive Roles 89

4. Engineering and Architecture Roles 90

5. Governance, Risk, and Compliance Roles 90

6. How the Roles Work Together 91

7. Certifications at a Glance 92

Section 12: Your Learning Roadmap 95

1. How This Program Is Organized 96

2. The Foundations, and Why They Come First 97

3. Choosing a Path 97

4. Building Skills in the Home Lab 98

5. Practising: CTFs and Challenges *98*

6. Building a Portfolio *99*

7. Staying Current, and Staying Safe *99*

8. Your Next Step *100*

References ***104***

How to Use This Book

This book is the first course in a cyber security program that takes you from knowing nothing to being ready for professional work. It assumes no security, no programming, and no networking. It has 12 sections. Each section builds on the ones before it, so read them in order.

Every section follows the same plan:

- ***What You Will Learn*** *and* ***Before You Start*** *say what the section covers, and what you should already know.*
- ***Numbered topics*** *teach one idea each, in plain language, with concrete examples, tables, and diagrams.*
- ***Common Mistakes*** *collects the errors people actually make, so you can avoid them.*
- ***Summary***, ***Check Your Understanding***, *and* ***Practice*** *help you remember the section. Every check-question has an answer, and every practice has a worked example.*

This is a thinking course, not a typing course. *Security Fundamentals builds the mental model and the language of the field. The hands-on, keyboard skills, Linux, networking, scripting, and attacking and defending in a lab, come in the courses that follow. There is almost no code here, and you need no special software to read this book.*

The one rule, from the first page. *Everything in this program is for defending. Later courses teach real attacks, but you only ever try them on systems you own, or that you have clear written permission to test, such as the home lab you will build. Attacking anyone else's computer, phone, website, or account is a crime in Cambodia and almost every country. Section 9 covers the ethics and the law in full.*

How the examples work. *The book uses small, realistic situations to explain ideas, a business hit by ransomware, a clinic's data, a shop's customer list. These are illustrations, not a running story: each section stands on its own, so if you already know a topic, you can skip to the next. Every name, key, password, and figure in an example is made up.*

Getting the most from it. *Read actively: at each* ***Practice****, stop and do it before reading the worked example. Keep notes as you go; those notes are the start of the portfolio that section 11 and section 12 explain. And when you finish, move straight on to the next Foundation course, in order, this book's final section is your roadmap for everything that follows.*

Course mark icon from Material Symbols by Google, Apache License 2.0. NIST, CIS, ISO, OWASP, and MITRE ATT&CK are the trademarks of their respective owners, used here only to refer to their published work.
Cover photo: “Siem Reap Reflections (CAMBODIA-REFLECTION-ANGKOR WAT) III” by Chi King, CC BY 2.0, via Flickr. Cropped and toned.

1

Section 1

What Cybersecurity Is

IN THIS SECTION

1. A Real Incident: How One Email Locked a Business 2
2. What Cybersecurity Is 3
3. Why It Matters Now 3
4. What We Protect: Information and Systems 4
5. The Three Questions 5
6. Threats, Vulnerabilities, and Risk 5
7. Defense Is Layers 6
8. Security Is Everyone's Job 6
9. Why We Learn to Attack 7
10. Ethics and the Law: The One Rule 7
11. The Road Ahead 8

What You Will Learn

- *What cybersecurity is, in plain words, and what it protects.*
- *Why it matters now, for a business, a person, and a whole country.*
- *The three questions every security person asks: can the wrong people **read** it, **change** it, or **stop** it?*
- *What a threat, a vulnerability, and a risk are, and how they differ.*
- *Why defense is built in layers, and why security is everyone's job.*
- *Why we learn to attack: to defend well, you must understand how attacks work.*
- *The one rule you must never break: only test what you own, or have written permission to test.*

Before You Start

You need nothing but a computer, a web browser, and curiosity. No programming, no networking, and no security knowledge. This is the first lesson of the whole program; the courses that teach the keyboard skills, Linux, networks, and code come after it.

***The one rule, from the very first lesson:** everything in these courses is for defending. Later you will learn real attacks, but you try them **only** on systems you own, or that you have clear written permission to test, such as the practice lab you will build in a later course. Attacking anyone else's computer, phone, website, or account, even "just to look", can harm real people, and in Cambodia and almost every country it is a crime. Topic 10 returns to this rule, and lesson 9 is devoted to it.*

---

1. A Real Incident: How One Email Locked a Business

Start with a kind of attack that happens somewhere in the world every day.

A small business runs on computers: a tablet takes orders, a laptop handles payroll, files sit on a shared drive. One morning the laptop shows a full-screen message in red: "Your files are locked. Pay to get them back." The sales records, the staff details, the supplier list, none of them will open.

What happened? A week earlier, an employee received an email that looked like it came from the company's internet provider: "Your bill is overdue, open the attached invoice." They opened the attachment. It quietly installed a program that encrypted the files and waited.

Nobody picked a lock. Nobody broke a window. One email, one click, and a business lost access to everything it runs on. This kind of attack is called ***ransomware****, and it is one of the most common serious attacks today. It is a good first example because it shows the whole subject in one picture: something valuable (the files), a way in (a fake email), a weakness (an employee who clicked), and harm (a business that cannot work). Cybersecurity is the practice of understanding that picture and changing the ending.*

2. What Cybersecurity Is

Cybersecurity is protecting information and the systems that hold it from people who should not read it, change it, or stop it from working.

Break that sentence into its parts:

- ***Information*** *is anything worth protecting: sales records, a phone number, a password, a photo, a bank balance.*
- ***Systems*** *are the things that store, move, and use that information: laptops, phones, servers, apps, websites, and the networks between them.*
- ***People who should not*** *are the attackers: from a curious insider, to a criminal who locks files for money, to a team paid by a government.*

An older name is ***information security****, or* ***infosec****. People use "cybersecurity", "information security", and simply "security" to mean much the same thing. In this program we usually say* ***security****.*

Security is not a product you buy once, and it is not a single lock. It is an ongoing practice: understanding what could go wrong, deciding what is worth protecting, putting defenses in place, and watching for the attacks that get through.

3. Why It Matters Now

Thirty years ago, a business kept its records on paper in a drawer. To steal them, you had to be in the room. Today those same records sit on a computer connected to the whole world, and an attacker can reach them from another country, at no cost, while you sleep.

Almost everything now runs on computers and networks: shops, banks, hospitals, schools, phones, cars, power stations. That makes life easier, and it makes a new kind of harm possible:

What depends on computers	What an attack can do
A business's records and payroll	Lock the files; stop the business for days
A person's phone and messages	Read private chats; steal money from a banking app
A bank or payment app	Move money; take over accounts
A hospital's records	Block access to patient information during an emergency
A city's power or water	Turn off a service that millions depend on

This is close to home. In Cambodia, as smartphones, online banking, and QR payments have spread, scams and account takeovers have grown with them. Security is no longer a concern only for large foreign companies. It matters for a small business, for every person with a phone, and for the country.

*Security is about **risk**, not perfection. No system is ever perfectly safe, and trying to make one perfect would cost more than it is worth. The useful questions are always: what could go wrong, how likely is it, how bad would it be, and what does it cost to prevent? A coffee shop needs less protection than a bank. Neither needs zero.*

4. What We Protect: Information and Systems

*Before you can defend something, you have to know what is worth defending. Security people call these things **assets**. An asset is anything that has value to you, and therefore might have value to an attacker.*

Think about a dental clinic. Its assets include:

Asset	Why it has value
Patient records	Private and legally protected; valuable to steal or hold for ransom
The appointment system	If it goes down, the clinic cannot see patients
The payment and banking details	Direct access to money
Staff accounts and passwords	A way in to everything else
The clinic's reputation	Patients who feel unsafe go elsewhere

Notice the last one. It is not a file or a machine. ***Trust and reputation are assets too.*** *A clinic can restore locked files from a backup in an hour, but winning back a patient who heard "that clinic leaked my records" can take far longer.*

Knowing your assets is the first step of every security decision, because you cannot protect everything equally. You protect what matters most, first.

5. The Three Questions

When a security person looks at any system, they ask three questions. These three are the heart of the subject, and the next lesson studies them in depth as the ***CIA triad****. In plain words:*

1. ***Can the wrong people _read_ it?*** *Keeping information from people who should not see it is called* ***confidentiality****. Example: only your bank and you should be able to read your account balance.*
2. ***Can the wrong people _change_ it?*** *Keeping information correct, and changed only in the right way, is called* ***integrity****. Example: nobody should be able to change the price on an online order from $100 to $1.*
3. ***Can the wrong people _stop_ it?*** *Keeping a system working when people need it is called* ***availability****. Example: a hospital's records must open during an emergency.*

These three give you a quick way to describe almost any attack. The ransomware in topic 1 was an ***availability*** *attack: the files were not read or changed, they were locked, so the business could not use them. Quietly copying a customer list to sell is a* ***confidentiality*** *attack. Giving yourself a discount you did not earn is an* ***integrity*** *attack. Almost every idea in this whole program protects one of these three: read, change, or stop.*

6. Threats, Vulnerabilities, and Risk

Three words get used constantly, and mixing them up causes confusion. They are different things.

- *A* ***threat*** *is something bad that _could_ happen, or someone who could make it happen. "A criminal could lock our files for money" is a threat.*
- *A* ***vulnerability*** *is a weakness that would let the bad thing happen. "Staff open email attachments without checking" is a vulnerability.*
- *A* ***risk*** *is the chance of the threat meeting the vulnerability, together with how bad the result would be. "It is fairly likely a staff member opens a bad attachment, and it would stop the business for days" is a risk.*

A simple way to hold them together:

> ***Risk = how likely × how bad.***

A threat with no matching vulnerability is not much of a risk. A locked door is a threat to someone with the wrong key, but not to someone with the right one. And a vulnerability that no threat targets may sit harmlessly for years, until one day it does not. Security work is finding the risks that are both likely and serious, and reducing them first.

*Lesson 3 rates and manages risk properly. For now, keep the three words apart: a **threat** is the bad event, a **vulnerability** is the weakness, and a **risk** is the chance and the cost of the two meeting.*

7. Defense Is Layers

The business in topic 1 did not have to lose its files. Suppose it had done a few ordinary things:

1. ***Training**, so the employee paused at a strange invoice email.*
2. ***An email filter**, that quietly moved the message to junk.*
3. ***Up-to-date software**, so the attachment's trick no longer worked.*
4. ***A separate account** for daily work, so the locking program could not reach every file.*
5. ***A backup**, made every night and kept disconnected, so the files could be restored in an hour.*

*Now the attacker has to get past all five. Miss the email filter? Training stops the click. Click anyway? The backup saves the day. This is **defense in depth**: several layers of protection, so that one failure does not become a disaster.*

No single defense is perfect. People get tired and click. Software has bugs. Filters miss things. Because every layer can fail, we never rely on just one. A good security plan assumes each defense will sometimes fail, and places another one behind it. Lesson 8 returns to this idea with the different kinds of control.

8. Security Is Everyone's Job

It is tempting to think security is one expert's job, or one product you install. It is neither. In the ransomware example, the weak point was not a machine. It was an ordinary email and an ordinary click.

Everyone involved has a part:

Who	*Their part in security*
The owner or manager	*Decides what matters, pays for backups and training, sets the rules*
The staff	*Spot strange emails, use good passwords, follow the rules*
The developer	*Builds software that does not leak data or trust the wrong things*
The server / IT	*Checks every request and owns the important decisions*
The security specialist	*Finds weaknesses, watches for attacks, and responds when one gets through*

A common and dangerous mistake is for each part to assume another is handling it: "the developer made it secure, so we're fine", or "we have antivirus, so we don't need backups". Security holds only when each part does its own job, and does not assume the others cover for it.

9. Why We Learn to Attack

This program teaches you how attacks work, and how to carry them out, in a safe lab. That surprises some people. Why learn to attack, if the goal is to defend?

*Because **you cannot defend against what you do not understand.** If you have never seen how a file gets locked, how a password is cracked, or how a fake email is built, your defenses are guesses. The people who protect banks and governments are the same people who know, in detail, how to break in. They have simply chosen to use that knowledge to defend, and to use it only where they are allowed.*

Think of a locksmith. To fit good locks, a locksmith must understand how locks are picked. The skill is the same; what differs is permission and intent. A locksmith who picks a lock because the owner hired them is doing their job. The same person picking a stranger's door is a burglar.

So in these courses you will learn real attacks: reading a password out of a program, intercepting network traffic, taking over a practice server. You will do all of it on systems built for the purpose. Which brings us to the rule that makes all of this safe and legal.

10. Ethics and the Law: The One Rule

Here is the rule again, because it matters more than any technique in this program:

> ***Only ever test systems that you own, or that you have clear, written permission to test.***

Everything you will learn, every attack and every tool, is powerful. Used on your own lab, or on a system whose owner has hired you, it is a valuable skill that pays well. Used on anyone else's system without permission, the exact same action is a crime, whatever your reason.

- *"I was only curious" is not permission.*
- *"The website was easy to break into" is not permission.*
- *"I didn't change anything, I only looked" is not permission.*
- *"I found a bug and wanted to help" is not permission, unless the owner invited you to look.*

Getting permission properly, scope, rules of engagement, and bug bounty programs, is covered in lesson 9 and again in the offensive courses. Until then, the safe rule is simple: your own lab, or written permission. There is no third option. A skilled person who respects this rule is a professional. One who does not is a criminal, and the skill is exactly the same.

11. The Road Ahead

This is one of 43 courses. They fit together like this:

Stage	*Courses*	*What you gain*
Foundations	*Security basics, how computers work, Linux, networking, Windows, scripting, Python, a home lab, cryptography*	*The ground every security person stands on*
Offensive	*Web, mobile, network, and wireless attacks, reverse engineering, exploitation, red teaming*	*How attacks really work, hands-on in the lab*
Defensive	*Security operations, incident response, forensics, malware analysis, threat intelligence*	*How to watch, catch, and recover from attacks*
Engineering	*Secure development, cloud, containers, identity*	*How to build systems that are hard to attack*
Specialized	*Industrial systems, fraud and banking, blockchain, AI security, governance, careers*	*Deep fields, and turning the skills into a job*

Everyone starts with Foundations, in order. After that you pick a path. You do not have to finish all 43 to be useful; after Foundations and one path, you are employable. But a complete security professional eventually touches all of them.

By the end of ***this course****, Security Fundamentals, you will understand the core ideas, speak the language, and be ready for the hands-on courses. The next lesson begins that work by studying the three questions in depth: the CIA triad and beyond.*

Common Mistakes

1. Thinking "we are too small to be a target." *Attackers often do not choose you on purpose. Automated attacks sweep the whole internet, and a small business with a weak defense is an easier catch than a bank.*

2. Buying one product and feeling safe. *Antivirus, or a firewall, or a password rule, is one layer. Security is many layers, plus people and habits.*

3. Confusing threat, vulnerability, and risk. *The bad event, the weakness, and the chance-and-cost of the two meeting are three different things. Keep them apart.*

4. Chasing perfect security. *No system is perfectly safe. Spend your effort on the risks that are both likely and serious.*

5. Thinking security is only the expert's job. *The weakest point is often an ordinary person and an ordinary click. Everyone has a part.*

6. Believing "learning to attack is wrong." *Learning how attacks work is how you defend. What matters is that you only ever practise on your own systems, or with written permission.*

7. Testing a system you do not own. *Curiosity is not permission. Only your own lab, or clear written permission. This one has no exceptions.*

Summary

Idea	Remember
Cybersecurity	Protecting information and systems from people who should not read, change, or stop them
Why it matters now	Everything runs on computers and networks; attacks reach you from anywhere
Risk, not perfection	Protect against what is likely and serious; nothing is ever perfectly safe
Assets	The things worth protecting, including trust and reputation
The three questions	Can the wrong people read it, change it, or stop it? (confidentiality, integrity, availability)
Threat / vulnerability / risk	The bad event / the weakness / the chance and cost of the two meeting
Defense in depth	Several layers, so one failure is not a disaster
Everyone's job	Owner, staff, developer, server, and specialist each have a part
Learn to attack	You cannot defend what you do not understand; permission and intent make it right
The one rule	Only test what you own, or have written permission to test

Check Your Understanding

1. In one sentence, what is cybersecurity?
2. A ransomware attack locks a business's files but does not read or steal them. Which of the three questions (read, change, stop) does that attack belong to?
3. Give one example each of a threat, a vulnerability, and a risk, different from the ones in the lesson.
4. Why is "trust and reputation" called an asset, when it is not a file or a machine?
5. What is defense in depth, and why do we use it instead of one strong defense?
6. A friend says, "learning to hack is just learning to be a criminal." How would you answer?
7. State the one rule for testing systems, and explain why "I was only curious" does not meet it.

Answers

1. Cybersecurity is protecting information and the systems that hold it from people who should not read it, change it, or stop it from working.
2. Availability: the files were not read or changed, but the business could not use them when it needed them.

3. *Any sensible set, for example: threat, "someone could steal an employee's phone"; vulnerability, "the phone has no screen lock"; risk, "phones are often lost, and this one holds the company's banking app, so the risk is high".*
4. *Because it has value and an attacker can damage it. If customers hear their data leaked, they stop trusting the business, which costs it money, even though no machine was broken.*
5. *Several layers of defense, so that if one fails, another still stops the attack. We use it because every single defense can fail: people click, software has bugs, filters miss things.*
6. *Learning how attacks work is how you build real defenses, just as a locksmith must understand lock-picking to fit good locks. What makes it right or wrong is permission and intent: we only ever practise on our own systems, or with written permission.*
7. *Only test systems you own, or have clear written permission to test. Curiosity is not permission: the harm to the owner, and the law, do not change because you were curious or meant no harm.*

Practice

Think about a system you use every day: your phone, a messaging app, your school or work email, or an online bank.

1. *List three **assets** it holds or protects that would matter to you if an attacker reached them.*
2. *For each asset, say which of the three questions worries you most: could the wrong people **read** it, **change** it, or **stop** it?*
3. *Name one **layer** of defense that already protects it (for example a screen lock, a password, two-step sign-in), and one more layer you could add.*

There is no single right answer. The goal is to practise looking at an everyday system the way a security person does: what is valuable, what could go wrong, and what layers stand in the way.

A Worked Example

For a phone with a banking app:

1. *Assets: the money in the banking app; private messages; saved photos.*
2. *Worries: for the banking app, that the wrong people could **change** it (move money) or **stop** it; for messages and photos, that the wrong people could **read** them.*
3. *A layer already there: the phone's screen lock and the app's sign-in. A layer to add: two-step sign-in (a code as well as a password) on the banking app, so a stolen password alone is not enough.*

2

Section 2

The CIA Triad and Beyond

IN THIS SECTION

1. Why We Need Clear Goals *13*

2. Confidentiality *13*

3. Integrity *14*

4. Availability *15*

5. Authentication, Authorization, and Accounting (AAA) *15*

6. Non-Repudiation *16*

7. The Trade-Offs Between Them *16*

8. Applying the Goals *17*

What You Will Learn

- *Why security needs clear, named goals.*
- *The three core goals: confidentiality, integrity, and availability.*
- *Three more goals that complete the picture: authentication, authorization, and accounting (AAA).*
- *What non-repudiation means, and why it matters for money and contracts.*
- *Why the goals pull against each other, and how to balance them.*

Before You Start

Read lesson 1 first. It introduced the "three questions": can the wrong people read, change, or stop something. This lesson gives those questions their proper names and adds the goals that sit around them. No programming or networking is needed.

---

1. Why We Need Clear Goals

"Make it secure" is not a plan. Secure against what? Readable by whom? Working how often? Without clear goals, two people can both say a system is "secure" and mean completely different things, and neither can check whether it is true.

Security people solve this with a small set of named goals. When you can say which goal you are protecting, you can design for it, test it, and tell when it fails. The oldest and most useful set is three goals, known together as the ***CIA triad****: confidentiality, integrity, and availability. (The name has nothing to do with any intelligence agency; it is just the first letters.)*

Most security work, in the end, is protecting one of these three. The rest of this lesson takes them one at a time, then adds the goals that support them.

2. Confidentiality

Confidentiality means only the right people can read the information.

It is what most people first think of as "security": keeping secrets secret. Your messages, your medical records, a company's customer list, a password, all should be readable only by those allowed to see them.

*Confidentiality is broken when information reaches someone who should not have it. That is called a **data breach** or a **leak**. It can happen many ways: a stolen laptop, a database left open on the internet, an employee emailing a file to the wrong person, or an attacker reading traffic on an open Wi-Fi network.*

The main tools for confidentiality are:

- ***Encryption:** scrambling data so that only someone with the key can read it. Lesson 9 of this course, and the whole Cryptography course, go deep on this.*
- ***Access control:** deciding who may open what (lesson 7).*
- ***Authentication:** proving who someone is before letting them in (topic 5).*

A quick test for confidentiality: if this fell into a stranger's hands, would it matter? If yes, it needs protecting.

3. Integrity

Integrity means the information is correct, and is changed only by the right people, in the right way.

Confidentiality is about reading; integrity is about changing. A bank balance that anyone could edit would be useless, even if nobody could read it. Integrity makes sure that what you store and send is what you get back, unchanged by accident or by an attacker.

*Integrity is broken when data is altered in a way it should not be: an attacker changing the amount of a payment, a virus corrupting files, or a simple transmission error flipping a bit. Note that integrity covers both **malicious** change (an attacker) and **accidental** change (a bug, a faulty disk).*

The main tools for integrity are:

- ***Hashing:** a short "fingerprint" of data; if the data changes, the fingerprint changes, so you can detect tampering (Cryptography course).*
- ***Digital signatures:** proof that data came from a particular person and was not changed on the way.*
- ***Access control and validation:** only allowing changes through checks that enforce the rules.*

A quick test for integrity: if someone changed this without permission, who would be harmed, and would we even notice?

4. Availability

Availability means the system works when the people who need it need it.

Information that is perfectly secret and perfectly correct is still useless if you cannot reach it. A hospital record you cannot open during an emergency has failed, even though nobody read it or changed it.

*Availability is broken by anything that stops legitimate use: a ransomware attack that locks files, a flood of fake traffic that overwhelms a website (a **denial-of-service** attack, covered in lesson 5), a power cut, a failed hard drive, or a cut network cable. Availability is the goal most often harmed by accidents and nature, not only attackers.*

The main tools for availability are:

- ***Backups:** copies you can restore from after loss or ransomware.*
- ***Redundancy:** spare parts, servers, and power, so one failure does not stop everything.*
- ***Capacity and filtering:** enough room to handle load, and defenses against floods of fake traffic.*

A quick test for availability: if this were unreachable for an hour, a day, a week, what would it cost?

5. Authentication, Authorization, and Accounting (AAA)

*The CIA triad says what we protect. Three more goals, often called **AAA**, say how we control who does what. They support all three parts of the triad.*

- ***Authentication** answers "who are you?" It is proving identity, usually with a password, a code, a fingerprint, or a passkey. Lesson 6 is devoted to it. Weak authentication breaks confidentiality (the wrong person reads data) and integrity (the wrong person changes it).*
- ***Authorization** answers "what are you allowed to do?" Once the system knows who you are, authorization decides which actions and data you may reach. A bank teller and a bank manager sign in the same way but may do different things. Lesson 7 covers this.*
- ***Accounting** (also called **auditing**) answers "what did you do?" It is the record, the **logs**, of who did what and when. Accounting does not stop an attack by itself, but it lets you detect one, investigate it, and prove what happened. The whole defensive side of security (SOC, incident response, forensics) depends on good logs.*

*A useful way to remember the order: you **authenticate** to prove who you are, the system **authorizes** what you can do, and it **accounts** for what you did.*

6. Non-Repudiation

Non-repudiation means someone cannot credibly deny that they did something.

*If you sign a contract, you should not be able to later claim you never did. In the digital world, non-repudiation is provided mainly by **digital signatures** and solid accounting (logs). Together they tie an action to a person strongly enough that the person cannot plausibly deny it, and an outsider can be convinced.*

This matters most for money, contracts, and legal records. When a customer authorizes a payment, the bank needs proof strong enough to stand up later, not just "our system says so". Non-repudiation builds on integrity (the record was not changed) and authentication (we know who acted).

7. The Trade-Offs Between Them

The goals pull against each other. Improving one often costs another. A good security design is a balance, chosen on purpose, not an attempt to maximize every goal at once.

Push this up...	***...and this can suffer***
Confidentiality *(lock everything down, extra passwords)*	***Availability*** *and ease of use: people are slowed or locked out*
Availability *(make everything easy to reach)*	***Confidentiality****: more ways in means more ways to leak*
Integrity *(check and approve every change)*	***Availability*** *and speed: more steps, more delay*
Security *in general*	***Usability*** *and* ***cost****: the classic trade-off*

Example: a bank could require five passwords and a manager's approval for every transaction. That is very strong on confidentiality and integrity, and terrible on availability. Nobody would use it. So the bank chooses a balance: strong sign-in, extra checks for large or unusual transactions, and smooth handling of ordinary ones.

*The right balance depends on what you are protecting, which is why lesson 1 said security is about **risk**. You spend protection where the asset is valuable and the threat is real, and you keep things usable everywhere else.*

8. Applying the Goals

The goals are a checklist for thinking about any system. Take an online shop:

Goal	A question to ask	A control that helps
Confidentiality	Who can read customers' addresses and card details?	Encryption, access control
Integrity	Can anyone change a price or an order total?	Server-side checks, signatures
Availability	What happens on the busiest sales day, or under attack?	Capacity, backups, anti-flood defense
Authentication	How do customers and staff prove who they are?	Passwords plus a second factor
Authorization	Can a customer reach another customer's order?	Role and ownership checks
Accounting	If money went missing, could we trace it?	Logs of every change
Non-repudiation	Can a customer deny a purchase they made?	Signed, logged transactions

You will not design perfect answers yet, that is what the rest of the program is for, but asking these seven questions about any system already puts you ahead of most people building them. In the next lesson, we turn to how we measure and manage the dangers these goals face: threats, vulnerabilities, and risk.

Common Mistakes

1. Treating "security" as one thing. *Saying a system is "secure" means nothing until you say which goal: readable by whom, changeable by whom, available how often.*

2. Forgetting availability. *Many people equate security with secrecy and forget that a system which does not work has also failed. Backups and redundancy are security.*

3. Thinking integrity is only about attackers. *A faulty disk or a bug corrupts data too. Integrity controls catch accidental change as well as malicious change.*

4. Confusing authentication and authorization. *Proving who you are is not the same as being allowed to do something. You can be correctly identified and still not permitted.*

5. Ignoring accounting until too late. *Without logs, you cannot tell what an attacker did, or prove what a user did. Set up logging before you need it.*

6. Trying to maximize every goal. *Pushing one goal to the extreme wrecks the others and the system's usability. Choose a balance on purpose.*

Summary

Term	Remember
CIA triad	Confidentiality, integrity, availability: the three core goals
Confidentiality	Only the right people can **read** it
Integrity	Information is correct and **changed** only in the right way
Availability	The system **works** when people need it
Authentication	Proving **who you are**
Authorization	What you are **allowed to do**
Accounting	The **record** of what was done (logs)
Non-repudiation	You **cannot deny** you did it
Trade-offs	The goals pull against each other; choose a balance based on risk

Check Your Understanding

1. What do the letters in "CIA triad" stand for?
2. A virus silently corrupts a file. Which goal has been broken?
3. A website is knocked offline by a flood of fake traffic. Which goal has been broken?
4. What is the difference between authentication and authorization?
5. What does accounting (auditing) give you that the other goals do not?
6. Give one example of non-repudiation mattering in real life.
7. Why can't a system simply maximize all the goals at once? Give one trade-off.

Answers

1. Confidentiality, Integrity, and Availability.
2. Integrity: the data was changed in a way it should not have been (even without an attacker reading it).
3. Availability: legitimate users cannot reach the site when they need it.
4. Authentication proves who you are; authorization decides what you are allowed to do once you are identified.

5. *A record of who did what and when (logs), so an attack or a user's actions can be detected, investigated, and proven after the fact.*
6. *Any sensible example: signing a contract and not being able to deny it; authorizing a bank payment that the bank can later prove you made.*
7. *Because the goals pull against each other. For example, pushing confidentiality up with many passwords and approvals reduces availability and usability, so nobody can get their work done.*

Practice

Pick one system you know well: a messaging app, an online bank, your email, or a school portal.

1. *Write one sentence for each of the three CIA goals, saying what it means for that system (for example, "Confidentiality: only I should read my messages").*
2. *For the same system, say how you **authenticate** (prove who you are), and name one thing you are **not authorized** to do.*
3. *Name one place where that system trades a goal away for usability (for example, staying signed in so you don't type a password every time, which trades some confidentiality for availability).*

A Worked Example

For an email account:

1. *Confidentiality: only I should read my emails. Integrity: nobody should be able to alter an email I received or send one as me. Availability: I should be able to reach my email when I need it.*
2. *I authenticate with a password and a code sent to my phone. I am not authorized to read another person's mailbox on the same mail service.*
3. *"Keep me signed in" on my own phone trades a little confidentiality (anyone holding my unlocked phone can read my mail) for availability and convenience (I don't re-enter my password each time).*

3

Section 3

Threats, Vulnerabilities, and Risk

IN THIS SECTION

1. The Three Words, Precisely 21

2. Where Vulnerabilities Come From 22

3. Threat Sources and Threat Events 22

4. Likelihood and Impact 23

5. Rating Risk: Qualitative and Quantitative 23

6. The Four Ways to Treat a Risk 24

7. Residual Risk 24

8. A Simple Risk Register 25

What You Will Learn

- The precise difference between a threat, a vulnerability, and a risk.
- Where vulnerabilities come from.
- The kinds of threat source, and what a threat event is.
- How to think about likelihood and impact.
- How to rate risk, simply (qualitative) and with numbers (quantitative).
- The four ways to treat a risk, and what residual risk means.
- How to write a simple risk register.

Before You Start

Read lessons 1 and 2. Lesson 1 introduced these three words loosely; this lesson makes them precise and shows how professionals use them to decide what to fix. No programming or networking is needed.

---

1. The Three Words, Precisely

These three words are the working language of risk. Using them precisely is the mark of someone who understands security.

- A **threat** is something bad that could happen, or an actor who could cause it. Example: a criminal group that spreads ransomware.
- A **vulnerability** is a weakness that a threat can use. Example: staff who open email attachments without checking.
- A **risk** is the combination: the chance that a threat meets a matching vulnerability, and how bad the result would be. Example: the risk of a ransomware outbreak that stops the business for days.

A fourth word ties them together. When a threat actually uses a vulnerability, that event is sometimes called an **exploit** (as a verb, "to exploit a weakness"; as a noun, the tool or method used). The attacker's path is: a **threat** uses an **exploit** against a **vulnerability**, and the **risk** is how likely that is and how much it would hurt.

Why be so careful? Because each word points to a different fix. You cannot usually remove the threat (criminals exist), but you can often remove the **vulnerability** (train staff, filter email), and you can always try to reduce the **risk** (add a backup so the impact is smaller). Naming the right

one tells you where to act.

2. Where Vulnerabilities Come From

Vulnerabilities are weaknesses, and they come from everywhere a system is built and used:

Source	Example
Software bugs	A coding mistake that lets an attacker run their own commands
Misconfiguration	A database left open to the internet with no password
Weak design	A system that trusts whatever the user's app tells it
People	Falling for phishing; reusing passwords; propping a door open
Process	No one applies security updates; nobody reviews who has access
Physical	An unlocked server room; an unattended, unlocked laptop
Third parties	A weakness in a library, supplier, or contractor you rely on

Notice that most vulnerabilities are not exotic hacking tricks. They are ordinary gaps: an unpatched program, a weak password, a helpful employee. Published software weaknesses get a public ID called a ***CVE*** *(Common Vulnerabilities and Exposures), so that everyone can track and fix the same issue; lesson 10's frameworks and the Vulnerability Management course build on this.*

3. Threat Sources and Threat Events

It helps to split a threat into two parts: who or what could cause harm (the ***threat source****), and the thing they do (the* ***threat event****).*

Threat sources *fall into a few groups (lesson 4 covers them in depth):*

- ***Adversarial:*** *people who mean harm, from lone opportunists to criminal groups to nation-states.*
- ***Accidental:*** *a user who deletes the wrong file, an admin who misconfigures a server.*
- ***Structural:*** *equipment that fails, software that crashes, a disk that dies.*
- ***Environmental:*** *fire, flood, power loss, an earthquake.*

This is a useful reminder that not every threat is a hacker. A flood that destroys your only server breaks availability just as surely as an attacker, and a good risk assessment includes both.

A **threat event** is the specific thing that happens: "an attacker guesses a weak password", "a disk fails", "a flood reaches the server room". Risk assessment pairs a source with an event with a vulnerability, and asks how likely and how bad.

4. Likelihood and Impact

Every risk has two dimensions:

- **Likelihood:** how probable is it that this threat event happens and succeeds? This depends on how exposed the vulnerability is, how attractive the asset is, and how capable the threat source is.
- **Impact:** if it does happen, how bad is it? Measured in money, downtime, harm to people, legal trouble, and damage to reputation.

Keeping these two apart is important because they call for different responses. A **high-likelihood, low-impact** risk (a staff member mistypes an order) you might simply accept or fix cheaply. A **low-likelihood, high-impact** risk (a fire destroys the office) you handle with insurance and off-site backups, even though it rarely happens. The dangerous ones are **high-likelihood, high-impact** risks, and those you fix first.

> **Risk = likelihood × impact.**

This simple formula is the heart of all risk work. It is why you do not spend the same effort on everything: you rank risks by likelihood times impact, and start at the top.

5. Rating Risk: Qualitative and Quantitative

There are two ways to put numbers, or at least order, on risk.

Qualitative rating uses simple bands like Low, Medium, and High, often drawn as a grid of likelihood against impact:

	Low impact	**Medium impact**	**High impact**
High likelihood	Medium	High	High
Medium likelihood	Low	Medium	High
Low likelihood	Low	Low	Medium

You place each risk in a cell and deal with the High cells first. It is quick, needs no exact data, and is good enough for most small organizations. Its weakness is that "High" is a judgement, and two people may disagree.

Quantitative *rating uses real money. A classic method estimates the yearly cost of a risk:*

- ***SLE (Single Loss Expectancy):*** *the cost of one occurrence. A stolen laptop: $1,500 to replace and recover.*
- ***ARO (Annualized Rate of Occurrence):*** *how many times a year it is expected. Laptops stolen: 2 per year.*
- ***ALE (Annualized Loss Expectancy) = SLE × ARO.*** *$1,500 × 2 = $3,000 per year.*

Now you can compare: if a laptop-encryption and tracking program costs $1,000 a year and cuts the loss in half, it saves $1,500 for $1,000, so it is worth doing. Quantitative rating is powerful for big decisions, but the numbers are only estimates, so do not mistake the precision for certainty.

Most real work mixes the two: qualitative to sort quickly, quantitative for the big or expensive decisions.

6. The Four Ways to Treat a Risk

Once you have ranked a risk, you choose how to treat it. There are only four options.

Treatment	Meaning	Example
Reduce (mitigate)	Add controls to lower the likelihood or impact	Train staff and filter email to cut phishing
Transfer	Make it someone else's cost, usually with insurance or a contract	Buy cyber-insurance for ransomware losses
Avoid	Stop doing the risky thing	Do not store card numbers at all; let a payment provider do it
Accept	Decide the risk is small enough to live with	Accept the rare mistyped order; it is cheap to fix

Two notes. First, ***accepting*** *a risk is a real, legitimate choice, as long as it is a decision, made by someone with the authority, and not just ignoring the risk. Second, you often combine treatments: reduce a risk with controls, transfer what remains with insurance, and accept the small part left over.*

7. Residual Risk

No treatment removes a risk completely. What is left after you have applied your controls is called ***residual risk****.*

Suppose phishing is a High risk. You add training, an email filter, and multi-factor authentication. The likelihood drops a lot, but not to zero, someone may still click, and a filter may still miss a clever message. That remaining chance is the residual risk. The organization's leaders then decide, explicitly, whether the residual risk is acceptable. If it is, they accept it; if not, they add more controls.

This is why security is never "finished". You reduce risk to an acceptable level, accept what remains, and keep watching, because both the threats and the systems change over time.

8. A Simple Risk Register

*Organizations keep their risks in a **risk register**: a simple table that lists each risk, its rating, its treatment, and who owns it. Even a one-page register, kept up to date, puts a small business ahead of most. Here is a small example:*

#	Risk	Likelihood	Impact	Rating	Treatment	Owner
1	Ransomware via phishing	High	High	**High**	Reduce: training, email filter, offline backups	Manager
2	Laptop lost or stolen	Medium	Medium	**Medium**	Reduce: encryption, screen lock; Transfer: insurance	IT
3	Website down on a busy day	Medium	High	**High**	Reduce: extra capacity, monitoring	IT
4	Staff mistypes an order	High	Low	**Low**	Accept; easy to correct	Staff lead
5	Fire in the office	Low	High	**Medium**	Transfer: insurance; Reduce: off-site backups	Owner

The register turns a vague worry ("are we secure?") into a short list of specific decisions, each owned by a real person. That is the whole point of risk work: not to remove every risk, which is impossible, but to see the risks clearly and decide what to do about each one. The frameworks in lesson 10 are, in large part, organized ways of doing exactly this.

Common Mistakes

1. Using the three words loosely. *A threat, a vulnerability, and a risk are different, and each points to a different fix. Say which one you mean.*

2. Trying to remove threats. You usually cannot remove the threat (criminals, fires, and failures exist). You remove vulnerabilities and reduce risk.

3. Treating every risk equally. Rank by likelihood times impact, and start at the top. Spending equally on everything wastes effort on risks that do not matter.

4. Thinking only attackers are threats. Accidents, hardware failure, and nature break the CIA goals too, and belong in the risk assessment.

5. "Accepting" a risk by ignoring it. Accepting is a deliberate decision by someone with authority, recorded in the register, not simply forgetting about it.

6. Believing security is ever finished. Residual risk always remains, and systems and threats change. Risk work is ongoing.

Summary

Term	Remember
Threat	A bad thing that could happen, or an actor who could cause it
Vulnerability	A weakness a threat can use
Risk	Likelihood × impact of a threat meeting a vulnerability
Threat source / event	Who or what causes harm / the specific thing that happens
Likelihood and impact	How probable, and how bad; keep them apart
Qualitative / quantitative	Low-Med-High bands / money estimates (SLE, ARO, ALE)
Four treatments	Reduce, Transfer, Avoid, Accept
Residual risk	What is left after controls; it must be accepted on purpose
Risk register	A table of risks, ratings, treatments, and owners

Check Your Understanding

1. In one sentence each, define threat, vulnerability, and risk.
2. Give the two parts of the risk formula.
3. Name the four threat-source groups, and give an example of a non-attacker threat.
4. A disk fails on average once every five years, and each failure costs $2,000 to recover. What is the ALE?
5. What are the four ways to treat a risk? Give an example of each.
6. What is residual risk, and who should decide whether to accept it?

7. *What is a risk register, and why is even a simple one useful?*

Answers

1. *A threat is a bad thing that could happen (or an actor who could cause it); a vulnerability is a weakness a threat can use; a risk is the likelihood and impact of the two meeting.*
2. *Risk = likelihood × impact.*
3. *Adversarial, accidental, structural, and environmental. A non-attacker example: a flood or fire (environmental), or a failed disk (structural).*
4. *ARO = 1/5 = 0.2 per year; SLE = $2,000; ALE = 0.2 × $2,000 = $400 per year.*
5. *Reduce (add controls, e.g. training), Transfer (insurance), Avoid (stop doing the risky thing, e.g. don't store card numbers), Accept (live with a small risk, e.g. a rare mistyped order).*
6. *The risk that remains after controls are applied. Someone with authority (a manager or owner) decides, explicitly, whether to accept it.*
7. *A table listing each risk with its rating, treatment, and owner. Even a simple one turns a vague worry into a short list of specific, owned decisions.*

Practice

Think of a small organization you know: a shop, a clinic, a school club, or your own home network.

1. *Write down three risks it faces. For each, name the **threat** and the **vulnerability** separately.*
2. *Give each risk a qualitative rating (Low, Medium, or High) using likelihood and impact.*
3. *Choose one of the four treatments for each, and say who should own it.*

Put your answers in a small five-column table: risk, threat, vulnerability, rating, treatment. You have just written a risk register.

A Worked Example

For a home network:

Risk	Threat	Vulnerability	Rating	Treatment
Wi-Fi password guessed	*A neighbour or passer-by*	*A short, default Wi-Fi password*	*Medium*	*Reduce: set a long password*
Phone lost with banking app	*Theft or loss*	*No screen lock; app stays signed in*	*High*	*Reduce: screen lock + app PIN*
Old router never updated	*An attacker using a known bug*	*Firmware years out of date*	*Medium*	*Reduce: update, or replace the router*

4

Section 4

Who Attacks and Why

IN THIS SECTION

1. The Myth of the Hooded Hacker	*30*
2. Script Kiddies and Opportunists	*31*
3. Cybercriminals and Ransomware Crews	*31*
4. Insiders	*32*
5. Hacktivists	*32*
6. Nation-State and Advanced Persistent Threats	*32*
7. Motivations: Money, Information, Disruption, and Belief	*33*
8. What This Means for Defending	*34*

What You Will Learn

- *Why the "lone hooded hacker" picture is mostly wrong.*
- *The main kinds of attacker, from opportunists to nation-states.*
- *Why insiders are a special and often underrated threat.*
- *The motivations behind attacks: money, information, disruption, and belief.*
- *What knowing your likely attackers means for how you defend.*

Before You Start

Read lessons 1 to 3. This lesson puts faces to the "threat sources" from lesson 3, and connects them to the risks you would put in a register. No programming or networking is needed.

---

1. The Myth of the Hooded Hacker

Films show a lone genius in a dark room, in a hoodie, breaking into a bank in minutes by typing fast. Real attackers look nothing like this, and believing the myth leads to poor defenses.

The reality:

- *Most attacks are **not** targeted at you personally. They are automated, sweeping the whole internet for any weak system, the way a thief tries every car door in a street.*
- *Many serious attackers are **organized groups**, run like businesses, with staff, budgets, and office hours, not lone geniuses.*
- *Most break-ins use **ordinary weaknesses**, a reused password, an unpatched program, a convincing email, not movie-style wizardry.*
- *The slow, boring parts, researching a target, waiting, trying again, matter far more than fast typing.*

Knowing who really attacks, and why, lets you defend against the attacks you will actually face, instead of the ones in films. We will go through the main kinds of attacker, usually grouped by skill, organization, and motive.

2. Script Kiddies and Opportunists

*At the least-skilled end are **script kiddies**: people who use tools and attacks that others built, without deeply understanding them. The name sounds harmless, but they do real damage, because the tools are powerful and freely available.*

*Closely related are **opportunists**: attackers who are not after you specifically, but will take any easy target they find. Most automated internet attacks are opportunistic. They scan millions of addresses for a known weakness and attack whatever answers.*

- ***Skill:** low to moderate; they rely on ready-made tools.*
- ***Targets:** whoever is easy; no particular victim.*
- ***Motive:** curiosity, showing off, small money, or just because they can.*
- ***Defense:** basic security stops most of them. Keep software updated, use strong unique passwords, and don't expose services you don't need. Opportunists move on to an easier target.*

*The key lesson: because so many attacks are opportunistic, **ordinary, basic defenses stop the majority of attacks**. You do not need to be unbreakable, only harder than the next target.*

3. Cybercriminals and Ransomware Crews

*The largest serious threat to most businesses is **organized cybercrime**: groups that attack for money, run like companies. Ransomware crews are the most visible today.*

*They operate as a real industry. Some groups write the ransomware and rent it to others (this is called **ransomware-as-a-service**). Others specialize in breaking in, or in laundering the money. They have support desks, negotiators, and profit targets. They often steal the data first and then lock it, so they can threaten to publish it even if you have backups (this is called **double extortion**).*

- ***Skill:** moderate to high, and well-resourced.*
- ***Targets:** anyone who can pay, from hospitals to small businesses; they pick by profit, not grudge.*
- ***Motive:** money, directly.*
- ***Defense:** the full defense-in-depth stack, offline backups above all, plus training, patching, and multi-factor authentication. The whole defensive side of this program exists largely to counter these groups.*

4. Insiders

*Not every attacker is outside. An **insider** is someone who already has legitimate access: an employee, a contractor, a supplier. Insiders are dangerous precisely because they are trusted and already inside your defenses.*

There are two kinds:

- ***Malicious insiders:** people who abuse their access on purpose, to steal data, to commit fraud, or out of anger (for example, a worker who copies the customer list before quitting to join a rival).*
- ***Accidental insiders:** people who cause harm by mistake, by far the more common, who fall for phishing, misconfigure a server, email a file to the wrong person, or lose a laptop.*

*Insiders are underrated because defenses usually point outward. The controls that help are **least privilege** (give each person only the access they need, lesson 7), **logging and monitoring** (accounting, lesson 2), separation of duties, and a culture where people feel able to report mistakes rather than hide them.*

5. Hacktivists

***Hacktivists** attack to make a political or social point. The word joins "hacker" and "activist". Their aim is usually attention, not money: defacing a website with a message, leaking documents they think the public should see, or knocking a service offline to protest an organization's actions.*

- ***Skill:** varies widely, from basic to capable.*
- ***Targets:** organizations they oppose: governments, companies, institutions in the news.*
- ***Motive:** belief, protest, publicity.*
- ***Defense:** the same fundamentals, with extra attention to anything public-facing (websites, social accounts) and to the ability to withstand a flood of traffic. Because the goal is attention, availability attacks and public defacement are common.*

6. Nation-State and Advanced Persistent Threats

*At the top end are **nation-state** attackers: teams funded by governments. In security their long campaigns are often called **APTs**, for **Advanced Persistent Threat**. The three words describe them well:*

- ***Advanced:** well-funded, highly skilled, sometimes using weaknesses nobody else knows about yet (called **zero-days**).*

- ***Persistent:*** *they do not give up after one try. They may stay hidden inside a network for months or years, pursuing a long-term goal.*
- ***Threat:*** *they have clear intent and real capability.*

Their motives are national: espionage (stealing secrets from governments and companies), sabotage (damaging an enemy's systems, including power and industry, the subject of the OT/ICS course), and influence.

- ***Targets:*** *governments, defense, large companies, infrastructure, and sometimes smaller companies used as a stepping-stone to a bigger one.*
- ***Defense:*** *very hard to stop completely. Most organizations cannot outspend a government. The realistic goals are to make yourself a costly target, to detect intrusions quickly (the SOC and threat-hunting courses), and to limit the damage.*

Most small businesses will never be a nation-state's direct target, but they can be caught in a ***supply-chain*** *attack, where the real target is a bigger customer reached through them.*

7. Motivations: Money, Information, Disruption, and Belief

Cutting across all these groups are a few basic motives. Knowing the motive helps you predict what an attacker wants and how they will behave.

Motive	*What they want*	*Typical attacker*
Money	*Direct payment, fraud, stolen accounts, resold data*	*Cybercriminals, ransomware crews*
Information	*Secrets, designs, personal data, credentials*	*Nation-states, competitors, data thieves*
Disruption	*To stop a service, damage, or cause chaos*	*Hacktivists, nation-states, disgruntled insiders*
Belief	*To make a point, protest, or gain attention*	*Hacktivists*
Ego / curiosity	*To prove it can be done, or to learn*	*Script kiddies, some opportunists*

Most attacks you will meet in business are about ***money****. That is why ransomware and fraud dominate, and why the defensive courses spend so much time on them.*

8. What This Means for Defending

You cannot defend equally against everyone, so you defend against the attackers you are actually likely to face. This is sometimes called knowing your ***threat model****.*

- *A* ***small shop*** *mostly faces opportunists and ransomware crews. Basic hygiene, updates, strong passwords, MFA, and offline backups, defeats the large majority of what it will meet.*
- *A* ***hospital or bank*** *faces the same, plus targeted criminals and, for a bank, fraud rings. It needs the full defensive stack and strong monitoring.*
- *A* ***government department or defense contractor*** *faces nation-states as well, and must plan for persistent, skilled attackers and quick detection.*
- ***Every*** *organization faces insiders, so least privilege and logging matter everywhere.*

The encouraging news from this lesson is that most attacks are opportunistic and use ordinary weaknesses, so ordinary, well-applied defenses stop most of them. You rarely need to be unbreakable. You need to be a harder, less profitable target than the next one, and to detect and recover quickly when something does get through. The rest of this course builds those ordinary defenses, and the paths that follow make them deep.

Common Mistakes

1. Defending against the movie hacker. *Real attacks are mostly automated, opportunistic, and use ordinary weaknesses. Defend against those first.*

2. "We're too small to be targeted." *Opportunistic attacks don't choose you; they find you. Small organizations are attacked constantly, precisely because they are soft.*

3. Ignoring insiders. *Most defenses point outward, but trusted people, usually by accident, cause a large share of incidents.*

4. Assuming every attacker is a genius. *Many use ready-made tools. Basic defenses stop them. Save your worry for the threats your organization actually faces.*

5. Planning to stop nation-states as a small business. *You usually cannot outspend a government. Aim to be costly to attack, quick to detect, and able to recover.*

6. Forgetting the motive. *Knowing whether an attacker wants money, information, or attention tells you what they'll go for and how they'll behave.*

Summary

Attacker	Skill	Motive	Main defense
Script kiddies / opportunists	Low	Curiosity, small gain	Basic hygiene; be harder than the next target
Cybercriminals / ransomware	Moderate–high	Money	Defense in depth; offline backups
Insiders	Any (they're trusted)	Money, anger, or accident	Least privilege; logging; culture
Hacktivists	Varies	Belief, publicity	Protect public-facing systems; withstand floods
Nation-states / APTs	Very high	Espionage, sabotage	Detect fast; limit damage; be costly to hit

Check Your Understanding

1. Give two ways the real picture of attackers differs from the movie "hooded hacker".
2. Why do basic defenses stop most opportunistic attacks?
3. What is double extortion, and why does it defeat backups alone?
4. Name the two kinds of insider, and say which is more common.
5. What do the three words in "Advanced Persistent Threat" each mean?
6. Match these motives to a likely attacker: money; information; publicity.
7. A small shop asks what it should worry about most. What do you tell it?

Answers

1. Any two: most attacks are automated and opportunistic, not targeted; many serious attackers are organized groups, not lone geniuses; most break-ins use ordinary weaknesses, not movie wizardry.
2. Because opportunists go after whoever is easy, not a specific victim. If you are harder than the next target, they move on. Updates, strong passwords, and not exposing needless services defeat most of them.
3. Stealing the data before locking it, so the attacker can threaten to publish it. Backups let you restore, but they do not stop the threat to leak the stolen copy.
4. Malicious insiders (abuse access on purpose) and accidental insiders (cause harm by mistake). Accidental is far more common.

5. *Advanced: well-funded and highly skilled, sometimes using unknown weaknesses. Persistent: they don't give up, and may hide for months or years. Threat: real intent and capability.*
6. *Money → cybercriminals/ransomware crews; information → nation-states (or competitors); publicity → hacktivists.*
7. *Opportunists and ransomware. Basic hygiene, updates, strong unique passwords, MFA, and offline backups, defeats the large majority of what a small shop will face.*

Practice

Think about one organization: your workplace, your school, or a small business you know.

1. *List the **three kinds of attacker** it is most likely to face, from this lesson.*
2. *For each, write the attacker's likely **motive** and **one defense** that would help.*
3. *Name one **insider** risk it has (malicious or accidental), and one control that would reduce it.*

A Worked Example

For a small online clothing shop:

1. *Opportunists; ransomware crews; (occasionally) hacktivists if it became controversial.*
2. *Opportunists, motive easy gain, defense: keep the website software updated. Ransomware crews, motive money, defense: offline nightly backups and staff training. Hacktivists, motive publicity, defense: protect the public website and social accounts, and be able to withstand a traffic flood.*
3. *Insider risk: a staff member accidentally emailing the customer list to the wrong person. Control: least privilege (few people can export the full list) plus logging of exports.*

5

Section 5

Common Attacks: Phishing, Malware, and More

IN THIS SECTION

1. How Most Attacks Actually Start 38

2. Phishing and Social Engineering 39

3. Malware: Viruses, Worms, Trojans, and Ransomware 40

4. Password Attacks 40

5. Attacks on Web and Network Services 41

6. Denial of Service 41

7. Man-in-the-Middle 42

8. The Attack Lifecycle 42

What You Will Learn

- *How most real attacks actually begin.*
- *Phishing and social engineering: attacking people, not machines.*
- *The main kinds of malware, and how they differ.*
- *How passwords are attacked.*
- *Attacks on web and network services, at a high level.*
- *Denial-of-service and man-in-the-middle attacks.*
- *The attack lifecycle: the stages most attacks move through.*

Before You Start

Read lessons 1 to 4. This lesson names the attacks the earlier lessons kept referring to. It is a map, not a manual: each attack is explained so you can recognize and defend against it, and each is studied in depth, safely and legally, in a later course. Remember the one rule: these are only ever tried on systems you own or have written permission to test. No programming is needed.

---

1. How Most Attacks Actually Start

Attacks are not usually a single dramatic break-in. They start small and quiet, most often in one of three ways:

1. ***A person is tricked** into clicking, opening, or telling (phishing and social engineering).*
2. ***A weakness is exploited** in software that is exposed to the internet and not updated.*
3. ***A stolen or guessed password** is used to simply log in.*

Study after study of real breaches finds the same thing: the large majority begin with one of these three, and very often with the first. The attacker does not need to be a genius; they need one person to click, or one unpatched server, or one reused password. This is good news for defenders, because it means a few basic defenses, aimed at these three doors, stop most attacks.

The rest of this lesson walks through the common attacks, grouped by what they target: people, machines, passwords, services, and the network.

2. Phishing and Social Engineering

Social engineering *is attacking the person instead of the computer: tricking someone into doing something that helps the attacker. It works because it is far easier to fool a human than to break strong encryption.*

Phishing *is the most common form: a fake message, usually email, that pretends to be from someone you trust, to make you click a link, open an attachment, or hand over a password. Variants have their own names:*

- ***Spear phishing:*** *a targeted phish, personalized for one person or company, often after research.*
- ***Whaling:*** *spear phishing aimed at an important person, like a CEO.*
- ***Smishing*** *and* ***vishing:*** *the same idea by SMS text and by voice call.*
- ***Business email compromise (BEC):*** *posing as a boss or supplier to get an employee to pay a fake invoice or change bank details. It steals enormous sums with no malware at all.*

How to recognize a phish: a sense of urgency ("act now or your account closes"), a request for a password or payment, a slightly wrong sender address, a link that does not go where it claims, and spelling or tone that is a little off.

Defenses: *training and healthy suspicion; checking the real sender address; never acting on urgency without verifying through another channel; email filtering; and multi-factor authentication, so a stolen password alone is not enough. The Social Engineering course covers this in depth, including running authorized phishing tests.*

3. Malware: Viruses, Worms, Trojans, and Ransomware

Malware is any software written to do harm. The word covers several kinds, grouped by how they spread and what they do:

Type	How it behaves
Virus	Attaches to a file or program; spreads when that file is run or shared by a person
Worm	Spreads by itself across a network, with no human action, by exploiting weaknesses
Trojan	Pretends to be something useful (a free game, a cracked app) so you install it yourself
Ransomware	Locks or encrypts your files and demands payment
Spyware	Secretly watches you: keystrokes, screens, passwords
Rootkit	Hides deep in the system to avoid detection and keep control
Botnet (bot)	Turns your machine into one of thousands the attacker controls remotely

These categories overlap: one real piece of malware can be a trojan that installs spyware and later ransomware. What matters for defense is less the label than the behavior.

Defenses: keep software updated (worms spread through known, unpatched holes); do not run untrusted programs or attachments; use anti-malware tools; limit accounts to least privilege so malware cannot reach everything; and keep offline backups so ransomware cannot destroy your only copy. The Malware Analysis course studies how malware works and how to analyze it safely.

4. Password Attacks

Because passwords guard so much, attacking them is a whole category. The main methods:

- **Guessing / brute force:** trying many passwords until one works. Automated tools try millions.
- **Dictionary attack:** trying common passwords and real words first, because people pick them.
- **Credential stuffing:** taking username-and-password pairs leaked from one breached site and trying them on others, because people reuse passwords. This is one of the most effective attacks today.
- **Password spraying:** trying one or two very common passwords against many accounts, to avoid locking any single one.
- **Phishing for passwords:** simply asking, via a fake login page.

Attackers rarely guess against a live login screen, which is slow and noisy. More often they steal a file of stored passwords and attack it offline. This is why how passwords are stored matters so much: they should be protected with slow ***password hashing****, never stored as plain text. Lesson 6 is devoted to passwords and authentication, and the Cryptography course explains hashing fully.*

Defenses: *long, unique passwords (a password manager makes this easy); multi-factor authentication, which defeats stolen passwords; and, on the defender's side, storing passwords with strong, slow hashing and watching for spraying and stuffing.*

5. Attacks on Web and Network Services

Any service exposed to the internet, a website, an app's back end, a mail server, is a door an attacker can knock on. The most common web attacks (studied hands-on in the Web Application Security course) include:

- ***Injection, such as SQL injection:*** *sending crafted input that the server mistakes for a command, letting an attacker read or change the database.*
- ***Cross-site scripting (XSS):*** *sneaking malicious code into a web page so it runs in other users' browsers.*
- ***Broken access control:*** *reaching data or actions you should not be allowed to, such as viewing another user's account by changing a number in the address.*

The common thread is ***trusting input that should not be trusted****, the same lesson as "never trust the client" from the mobile world. The defense, broadly, is to treat all input as untrusted, validate it, and make the server, not the user, enforce the rules. The OWASP Top 10 (lesson 10, and the web course) is the standard list of these risks.*

6. Denial of Service

A ***denial-of-service (DoS)*** *attack does not steal or change data; it attacks* ***availability****, by overwhelming a system so real users cannot use it. The usual method is a flood: far more requests than the system can handle, so it slows to a crawl or falls over.*

A ***distributed denial-of-service (DDoS)*** *attack sends the flood from many machines at once, often a* ***botnet*** *of thousands of infected devices around the world. This makes it bigger and much harder to block, since the traffic comes from everywhere.*

Defenses: *extra capacity; filtering and rate-limiting; and specialized anti-DDoS services that absorb and scrub the flood before it reaches you. For most small organizations, a hosting provider or a content-delivery network handles the worst of it.*

7. Man-in-the-Middle

A **man-in-the-middle (MITM)** attack puts the attacker secretly between you and who you think you are talking to, so they can read or change the messages passing through. A classic setting is an open public Wi-Fi network, where an attacker on the same network can position themselves in the path of your traffic.

If the traffic is unencrypted, the attacker reads everything, and can alter it. This is exactly why **HTTPS** (the padlock, encrypted web traffic) matters: it encrypts the connection so a man in the middle sees only scrambled data, and it checks that you are really connected to the right site, not an impostor. The Networking and Cryptography courses explain how this protection works.

Defenses: use encrypted connections everywhere (HTTPS, a VPN on untrusted Wi-Fi); heed browser certificate warnings instead of clicking past them; and, for apps, verify the server's identity properly.

8. The Attack Lifecycle

Most serious attacks are not one action but a sequence of stages. Security people use models of this sequence, such as the **Cyber Kill Chain** and the more detailed **MITRE ATT&CK** (lesson 10), to understand and interrupt attacks. A simplified lifecycle:

1. **Reconnaissance:** research the target, find email addresses, systems, and weaknesses.
2. **Initial access:** get in, usually via phishing, an exposed weakness, or a stolen password.
3. **Establish foothold:** install something to keep access, even if the first way in closes.
4. **Escalate privileges:** go from a normal account to an administrator (the Pentest courses cover this).
5. **Move laterally:** spread from the first machine to more valuable ones.
6. **Act on the objective:** steal data, deploy ransomware, commit fraud, whatever the goal was.

Why this matters: the attacker must succeed at **every** stage, but the defender only has to stop **one**. This is defense in depth again (lesson 8). Catch the phish and you stop stage 2. Apply least privilege and you block stage 4. Monitor the network and you spot stage 5. Keep offline backups and you blunt stage 6. Every layer is a chance to break the chain.

Common Mistakes

1. Thinking attacks are high-tech wizardry. Most start with a tricked person, an unpatched system, or a stolen password. Defend those three doors first.

2. Judging a phish by spelling alone. *Modern phishing can be flawless. Rely on verifying the sender and the request through another channel, not just on typos.*

3. Treating all malware as "a virus". *Worms spread on their own; trojans trick you into installing them; ransomware extorts you. The defenses differ, and behavior matters more than the label.*

4. Reusing passwords. *Credential stuffing exists precisely because people reuse passwords. One breached site then unlocks all the others.*

5. Trusting open Wi-Fi. *Without encryption, a man in the middle reads everything. Use HTTPS and a VPN on untrusted networks.*

6. Thinking you must stop every stage. *You don't. The attacker must win every stage; you only need to break the chain once. That is the power of layers.*

Summary

Attack	Targets	Core defense
Phishing / social engineering	People	Training, verify, MFA, email filtering
Malware (virus, worm, trojan, ransomware)	Machines	Updates, least privilege, anti-malware, offline backups
Password attacks	Credentials	Unique passwords, MFA, strong hashing on the server
Web/network service attacks	Services	Treat input as untrusted; server enforces rules
Denial of service (DoS/DDoS)	Availability	Capacity, filtering, anti-DDoS services
Man-in-the-middle	Traffic	Encryption (HTTPS/VPN); heed certificate warnings
Attack lifecycle	All stages	Defense in depth: break the chain at any stage

Check Your Understanding

1. What are the three most common ways attacks start?
2. What is the difference between phishing and spear phishing?
3. How does a worm differ from a virus?
4. What is credential stuffing, and which bad habit makes it work?
5. What does a denial-of-service attack target, and how does DDoS differ from DoS?
6. Why does HTTPS defend against a man-in-the-middle attack on open Wi-Fi?

7. *In the attack lifecycle, why is it said that the defender has the advantage?*

Answers

1. *A person tricked (phishing/social engineering); a software weakness exploited on an exposed, unpatched system; and a stolen or guessed password used to log in.*
2. *Phishing is a generic fake message sent widely; spear phishing is targeted and personalized for one person or company, often after research.*
3. *A virus attaches to a file and spreads when a person runs or shares it; a worm spreads by itself across a network with no human action, by exploiting weaknesses.*
4. *Taking username/password pairs leaked from one site and trying them on others. It works because people reuse the same password across sites.*
5. *Availability: it overwhelms a system so real users cannot use it. DDoS sends the flood from many machines at once (often a botnet), making it bigger and harder to block than a single-source DoS.*
6. *HTTPS encrypts the connection, so a man in the middle sees only scrambled data, and it verifies the site's identity, so you are not fooled by an impostor.*
7. *Because the attacker must succeed at every stage of the lifecycle, while the defender only has to stop one stage to break the chain.*

Practice

For each of the three common starting points of an attack, name one defense a small business could put in place this week.

1. *A person being tricked (phishing).*
2. *An exposed, unpatched system.*
3. *A stolen or reused password.*

Then pick one attack from this lesson and describe, in two or three sentences, how you would explain it to a non-technical shop owner, and the single most important thing they should do about it.

A Worked Example

1. *Phishing: a 30-minute staff talk on spotting urgent, payment-related emails, plus turning on the email provider's spam filtering.*
2. *Unpatched system: turn on automatic updates for the computers and the website software.*
3. *Stolen/reused password: turn on multi-factor authentication for email and banking, and give staff a password manager.*

Chosen attack, ransomware: "A criminal locks all your files and demands money. Even if you'd rather not pay, the safest protection is a backup made every night and kept disconnected from the computer, so you can restore everything and ignore the demand. Set that up first."

6

Section 6

Passwords and Authentication

IN THIS SECTION

1. What Authentication Is 47

2. The Three Factors 48

3. Why Passwords Fail 48

4. How Passwords Are Stored and Cracked 49

5. Making Strong Passwords and Passphrases 49

6. Password Managers 50

7. Multi-Factor Authentication 50

8. Passkeys: The Direction of Travel 51

What You Will Learn

- What authentication is, and the three factors it can use.
- Why passwords fail so often.
- How passwords are stored and cracked, and why storage matters.
- How to make strong passwords and passphrases.
- Why a password manager is one of the best security habits.
- What multi-factor authentication is, and why it defeats most attacks.
- What passkeys are, and why they are the direction of travel.

Before You Start

Read lessons 1 to 5. Lesson 2 introduced authentication as part of AAA; lesson 5 showed how passwords are attacked. This lesson is the practical core: the habits here protect you more than almost anything else you will learn. No programming is needed.

---

1. What Authentication Is

Authentication is proving that you are who you say you are. It is the lock on the front door of almost every system: before it lets you read or change anything, it checks your identity.

Keep two words apart, from lesson 2:

- **Identification** is claiming an identity ("I am Dara"), usually a username.
- **Authentication** is proving it ("and here is my password / fingerprint / code").

A username is not a secret; it is often an email address. The secret is the proof. So the whole strength of a login rests on how good that proof is, which is what this lesson is about.

2. The Three Factors

There are three fundamentally different kinds of proof, called **factors**:

Factor	Meaning	Examples
Something you know	A secret in your head	Password, PIN, the answer to a question
Something you have	A physical thing you possess	A phone, a security key, a bank card
Something you are	A physical trait of your body	Fingerprint, face, voice

Each has weaknesses. Something you **know** can be guessed, phished, or reused. Something you **have** can be lost or stolen. Something you **are** cannot be changed if it leaks, and can sometimes be copied.

The power comes from **combining** factors, which we return to in topic 7. If a login needs two different factors, an attacker must beat both, and beating a password is very different from stealing your phone. Using two of the same kind (two passwords) is not nearly as strong, because one attack, phishing, can take both.

3. Why Passwords Fail

Passwords are the oldest and most common factor, and the most troublesome. They fail for human and technical reasons:

- **People choose weak ones.** "123456" and "password" top every leaked-password list, year after year.
- **People reuse them.** The same password across many sites means one breach unlocks them all (credential stuffing, lesson 5).
- **People can be tricked into giving them away.** A password is just words, so a convincing fake login page collects it in seconds.
- **They can be guessed in bulk.** Computers try billions of guesses against a stolen password file.
- **We have too many.** A person may have hundreds of accounts, far more than anyone can remember with unique strong passwords, so they cut corners.

None of this means passwords are useless. It means a password alone is weak, and must be made strong (topics 5 and 6) and backed by a second factor (topic 7). The long-term answer, passkeys, removes the password entirely (topic 8).

4. How Passwords Are Stored and Cracked

To check your password, a system must store something to compare against. How it stores that something decides what happens when the store is stolen, and stores are stolen often.

- ***The worst way: plain text.*** *The password is saved as-is. If the file leaks, every password is instantly exposed. No competent system does this, but some still do.*
- ***The right way: hashing.*** *The system stores a* ***hash****, a scrambled fingerprint produced by a one-way function. You can turn a password into its hash, but you cannot turn the hash back into the password. At login, the system hashes what you typed and compares the two hashes.*

Hashing is not perfect on its own. Attackers who steal a file of hashes crack it ***offline****, guessing passwords, hashing each guess, and looking for a match, at billions of guesses a second. Two techniques make this much harder:*

- ***Salting:*** *adding a unique random value to each password before hashing, so identical passwords get different hashes and attackers cannot crack them all at once with precomputed tables.*
- ***Slow hashing:*** *using a function built to be deliberately slow (such as bcrypt, scrypt, or Argon2), so each guess costs real time, turning billions of guesses a second into a few thousand.*

You do not need the math yet; the Cryptography course covers it. The takeaway for now: ***a strong password matters because storage can be stolen, and good storage, salted slow hashing, matters because it buys time to respond even if it is.*** *This is also why a long password helps so much: each extra character multiplies the number of guesses an attacker must make.*

5. Making Strong Passwords and Passphrases

What makes a password hard to crack is not mostly strange symbols; it is ***length and unpredictability****. A short password full of symbols can be weaker than a long, simple one.*

- ***Length wins.*** *Every extra character multiplies the work to guess it. Aim for at least 12–16 characters where you must remember it.*
- ***Passphrases are strong and memorable.*** *Four or five random, unrelated words, like* `correct-harbor-tiger-bottle`*, are long, easy to recall, and very hard to guess, as long as the words are truly random and not a famous phrase.*
- ***Unpredictable, not just complex.*** `P@ssw0rd!` *looks complex but is one of the first things cracking tools try, because the substitutions are obvious. Randomness beats cleverness.*

- ***Unique per site.*** *Even a strong password is dangerous if reused, because one breached site exposes it everywhere.*
- ***Do not rely on forced "complexity rules".*** *Rules demanding a symbol and a number often push people to predictable patterns. Length and uniqueness matter more.*

Remembering a unique long password for every account is impossible by hand, which is exactly the problem a password manager solves.

6. Password Managers

*A **password manager** is an app that generates, stores, and fills in a different strong password for every account. You remember **one** strong master password (or unlock it with your fingerprint or face), and it remembers all the rest, encrypted.*

Why it is one of the best security habits you can adopt:

- *It makes **unique, long, random** passwords effortless, defeating reuse and guessing at once.*
- *It **fills passwords only on the correct site**, which quietly defends against phishing: if the site is a fake look-alike, the manager will not recognize it and will not fill in your password.*
- *It lets you **see and change** weak or reused passwords across all your accounts.*

*Common worries, answered: a reputable manager stores everything **encrypted**, so even its own company cannot read your passwords; and keeping them in one well-protected, encrypted vault is far safer than the realistic alternative, reusing a handful of weak passwords everywhere, or writing them on paper. Protect the manager itself with a strong master password and multi-factor authentication.*

7. Multi-Factor Authentication

***Multi-factor authentication (MFA)**, also called two-factor (2FA) or two-step verification, requires **two different factors** to log in: usually something you know (your password) plus something you have (a code on your phone, or a tap on a security key).*

It is the single most effective step most people and businesses can take, because it breaks the most common attacks. A stolen or guessed password is now useless on its own: the attacker also needs your second factor. Credential stuffing, most phishing, and bulk password cracking all fail against MFA.

Not all second factors are equal:

- ***SMS codes** (texted to your phone) are common and far better than nothing, but can be intercepted or redirected, so they are the weakest MFA.*

- ***Authenticator apps*** *that generate a changing 6-digit code are stronger and work offline.*
- ***Security keys*** *(small hardware devices) and* ***passkeys*** *(topic 8) are the strongest, because they also verify the site and cannot be phished.*

Turn MFA on everywhere it is offered, starting with the accounts that matter most: email (which can reset all your other passwords), banking, and work. One caution: some phishing now tries to trick you into approving an MFA prompt or handing over a code, so treat an unexpected prompt as a warning, never approve one you did not start.

8. Passkeys: The Direction of Travel

The newest and most promising answer removes the password altogether. A ***passkey*** *is a secret that lives on your device (your phone or computer) and proves your identity without ever sending a password to the site. You unlock it the way you unlock your phone, with your fingerprint, face, or PIN.*

Why passkeys are better:

- ***Nothing to phish.*** *There is no password to type, so there is nothing for a fake site to steal. The passkey also checks it is talking to the real site.*
- ***Nothing to reuse or leak in a breach.*** *Each passkey is unique to one site, and the site never stores a secret that could be stolen and cracked.*
- ***Easy to use.*** *Signing in is just unlocking your device, which is faster than typing a password.*

Passkeys are built on well-tested public-key cryptography (the Cryptography course explains how) and are supported by the major platforms and a growing number of websites. They are the direction the whole industry is moving. For now, the practical advice is layered: use a password manager for unique strong passwords, turn on MFA everywhere, and adopt passkeys wherever a site offers them. Lesson 7 continues the story: once a system knows who you are, how it decides what you are allowed to do.

Common Mistakes

1. Confusing identification with authentication. *A username is a claim, not a secret. The password (or key) is the proof, and it is what must be strong.*

2. Reusing passwords. *The biggest single password mistake. One breached site exposes every account that shares the password.*

3. Choosing complex over long. `P@ssw0rd!` *is weaker than a long passphrase. Length and unpredictability beat obvious symbol tricks.*

4. Storing passwords in plain text (as a builder). *If the store leaks, every password is instantly exposed. Always use salted, slow hashing.*

5. Refusing a password manager out of fear. *A reputable, encrypted manager is far safer than the reuse and weak passwords people fall back on without one.*

6. Treating all MFA as equal. *SMS is the weakest; authenticator apps, security keys, and passkeys are stronger. Any MFA still beats none.*

7. Approving an MFA prompt you did not start. *That is now a common attack. An unexpected prompt is a warning, not a convenience.*

Summary

Term	Remember
Authentication	*Proving you are who you claim to be*
The three factors	*Something you know / have / are*
Why passwords fail	*Weak, reused, phishable, guessable, too many*
Hashing / salting / slow hashing	*How passwords should be stored, so a stolen file resists cracking*
Strong passwords	*Long and unpredictable beats short and complex; unique per site*
Password manager	*One master password; unique strong passwords for all; anti-phishing*
MFA	*Two different factors; defeats stolen passwords; SMS weakest*
Passkeys	*Passwordless, unphishable, unique per site; the future*

Check Your Understanding

1. *What is the difference between identification and authentication?*
2. *Name the three factors, with an example of each.*
3. *Why is reusing a password so dangerous?*
4. *Why does it matter whether a site stores passwords as plain text or as salted, slow hashes?*
5. *Which is usually stronger: a short password with many symbols, or a long passphrase of random words? Why?*
6. *Give two reasons a password manager improves security.*
7. *Why does MFA defeat a stolen password, and which MFA method is the weakest?*

8. *In one sentence, why is a passkey hard to phish?*

Answers

1. *Identification is claiming an identity (a username); authentication is proving it (a password, code, or key). The proof is the secret part.*
2. *Something you know (password/PIN); something you have (phone/security key); something you are (fingerprint/face).*
3. *Because a breach of one site exposes that password, and attackers then try it on all your other accounts (credential stuffing). One leak unlocks many accounts.*
4. *If stored as plain text, a leak exposes every password instantly. Salted, slow hashing means a stolen file is slow and hard to crack, buying time to respond.*
5. *The long passphrase, usually. Length multiplies the guesses needed, while a short password's obvious symbol tricks are the first things cracking tools try.*
6. *Any two: it makes unique long random passwords effortless; it fills passwords only on the correct site, defending against phishing; it lets you find and fix weak or reused passwords.*
7. *Because the attacker also needs your second, different factor (such as your phone), which a stolen password does not provide. SMS codes are the weakest MFA.*
8. *A passkey never sends a password and verifies it is talking to the real site, so a fake site has nothing to capture and cannot impersonate the real one.*

Practice

A practical lesson deserves a practical exercise. Do these for real, on your own accounts:

1. *List your **three most important accounts** (usually email, banking, and one more).*
2. *For each, check whether **MFA** is turned on. If not, and it is offered, turn it on, starting with email.*
3. *Decide on a **password manager** to try, and plan to move your reused passwords into unique ones, most important accounts first.*

Then answer in writing: which single one of your accounts, if an attacker took it over, would let them reach the most other accounts? (For most people it is email, because it can reset other passwords, which is why it should get the strongest protection first.)

A Worked Example

1. *Most important: email, mobile banking, and a social media account tied to my real name.*
2. *Email: MFA was off, now on with an authenticator app. Banking: MFA already on via the bank app. Social media: turned on with an authenticator app.*

3. *Chose a reputable password manager; plan to change email and banking to unique 16-character generated passwords first, then work through the rest.*

The account that reaches the most others: email, because password resets for almost everything land there, so it gets an authenticator-app second factor and a unique strong password before anything else.

7

Section 7

Access Control Basics

IN THIS SECTION

1. Authentication vs. Authorization 56

2. Least Privilege 56

3. Separation of Duties 57

4. Access Control Models 58

5. Users, Groups, and Roles 58

6. Privilege Creep 59

7. Access Control in Practice 59

What You Will Learn

- The difference between authentication and authorization.
- The principle of least privilege, the most important idea in access control.
- Separation of duties, and why it stops fraud.
- The main access control models: DAC, MAC, RBAC, and ABAC.
- How users, groups, and roles organize access.
- Privilege creep, and why access must be reviewed.

Before You Start

Read lesson 6 (authentication) and lesson 2 (AAA). Authentication proved who you are; this lesson is about what you are allowed to do once you are known. No programming is needed.

---

1. Authentication vs. Authorization

These two words are often confused, and keeping them apart is the foundation of this lesson.

- **Authentication** answers "**who are you?**" (lesson 6). It happens once, at the door.
- **Authorization** answers "**what are you allowed to do?**" It happens on every action, after you are known.

An analogy: at an airport, authentication is showing your passport to prove who you are. Authorization is your boarding pass, it says which flight and seat you may take. Proving your identity does not, by itself, let you board any plane you like.

A real system checks both. You might be perfectly, correctly authenticated as yourself, and still not be authorized to read the payroll file or delete another user's account. A huge share of real breaches come from **broken authorization**: a user who is logged in as themselves reaches data or actions they should never have been allowed, often just by changing a number in a web address. Getting authorization right is as important as getting authentication right.

2. Least Privilege

The principle of least privilege: give each person, program, and device only the access it needs to do its job, and no more.

This is the single most important idea in access control, and one of the most important in all of security. If a user only needs to read orders, they should not be able to delete customers. If a program only needs to write to one folder, it should not be able to touch the whole disk.

Why it matters so much:

- ***It limits the damage of any single failure.*** *If an account with least privilege is phished or a program is hacked, the attacker gains only that small slice of access, not the keys to everything. Least privilege is what turns a disaster into an incident.*
- ***It limits mistakes.*** *People cannot accidentally delete what they cannot touch.*
- ***It shrinks the attack surface.*** *Fewer permissions means fewer things an attacker can abuse.*

A closely related idea is the practice of ***not using an administrator account for everyday work****. Even an IT administrator should browse the web and read email as an ordinary user, and switch to their powerful account only for the specific task that needs it, so that a bad click does not happen while holding the keys to everything. Least privilege applied in time, not just in scope.*

3. Separation of Duties

Separation of duties means no single person can complete a sensitive process alone. *A critical action is split so that it needs two or more people, which stops one person acting fraudulently or by mistake.*

The classic example is money: the person who requests a payment should not also be the person who approves it. One employee raises an invoice; a different one approves the funds. To commit fraud, you would now have to corrupt two people, which is far harder, and far more likely to be noticed.

Separation of duties shows up all over security:

- *Requesting versus approving access to a system.*
- *Writing code versus approving its release to production.*
- *An administrator who can change settings, but a separate person who reviews the logs of those changes.*

It pairs naturally with least privilege: least privilege limits how much power one role has, and separation of duties ensures the most dangerous powers are split across roles. Together they are a strong defense against both insider threats (lesson 4) and honest mistakes.

4. Access Control Models

*How does a system actually decide who may do what? There are a few standard **models**. You do not need to implement them yet, but you should recognize the names and the ideas.*

Model	How access is decided	Where you see it
DAC (Discretionary Access Control)	The **owner** of a resource decides who may use it	Files on your computer; sharing a document and choosing who can view it
MAC (Mandatory Access Control)	A **central policy** decides, based on labels (for example "Secret"); users cannot override it	Military and high-security systems
RBAC (Role-Based Access Control)	Access is granted to **roles**, and people are given roles	Most businesses: "Cashier", "Manager", "Admin"
ABAC (Attribute-Based Access Control)	Access is decided by **attributes and rules** (user's department, time of day, device, location)	Large, flexible modern systems

*The two you will meet most are **DAC**, the everyday model where owners share their own files, and **RBAC**, the backbone of almost every business system. **MAC** is strict and central, used where secrecy is paramount. **ABAC** is the most flexible and powerful, deciding access from a rich set of conditions, and is common in large cloud systems. Real systems often blend them.*

5. Users, Groups, and Roles

Managing permissions for each person one by one does not scale, and quickly becomes a mess. So access is organized in layers:

- *A **user** is one account, one person (or one program or device).*
- *A **group** is a bundle of users, so you can grant access to many people at once ("Phnom Penh Store Staff").*
- *A **role** is a named set of permissions tied to a job ("Cashier" can take payments and issue refunds up to a limit).*

*The power of this is that you manage access by **job, not by person**. You decide once what a "Cashier" may do, then give the Cashier role to whoever works the till. When someone changes jobs, you change their role, and their access follows. When a new weakness or rule appears, you fix it in one place, the role, and everyone with that role is updated. This is RBAC in practice, and it is how almost every organization keeps access manageable.*

6. Privilege Creep

Access is not a one-time decision; it rots over time. ***Privilege creep*** *(also called access creep) is the slow build-up of permissions a person no longer needs.*

It happens naturally: someone joins the sales team and gets sales access; they help the finance team for a month and get finance access; they move to support and get support access, but nobody ever removes the first two. After a few years, a long-serving employee may be able to reach almost everything, which quietly destroys least privilege and makes that one account a huge prize for an attacker.

The fixes are process, not technology:

- ***Access reviews:*** *regularly check who has access to what, and remove what is no longer needed. This is called* ***recertification****.*
- ***Joiner-mover-leaver process:*** *grant access when someone joins, change it when they move roles, and promptly remove all of it when they leave. Forgotten accounts of former staff are a classic way in.*
- ***Time-limited access:*** *grant temporary access that expires automatically, so it cannot linger.*

Privilege creep is a reminder that access control is an ongoing practice, the same lesson as residual risk in lesson 3: security is never "set and forget".

7. Access Control in Practice

Put the pieces together for a small business, say an online shop:

Idea	*In practice*
Authentication	*Everyone signs in with a unique account and MFA (lesson 6)*
Authorization	*The system checks, on every action, whether this role may do it*
Least privilege	*A cashier can process orders, but cannot change prices or export the customer list*
Separation of duties	*Refunds over a limit need a manager's approval; the person who requests is not the one who approves*
Roles (RBAC)	*"Cashier", "Manager", "Admin" roles, assigned by job*
Privilege creep	*Access is reviewed every quarter, and removed the day someone leaves*

Notice that none of this requires exotic technology. It requires clear thinking about who needs what, and the discipline to keep it current. Those habits prevent a large share of real incidents: an over-powered account that gets phished, a former employee whose login still works, a

cashier who can quietly export every customer's details. Access control is where the goals of lesson 2, especially confidentiality and integrity, are enforced in daily practice. In the next lesson, we widen the view to all the kinds of control a security program uses, and how they stack into defense in depth.

Common Mistakes

1. Confusing authentication and authorization. *Being correctly logged in as yourself does not mean you are allowed to do everything. The system must check permission on every action.*

2. Giving everyone broad access "to be safe". *The opposite is true. Broad access means any single compromised account is a catastrophe. Least privilege limits the blast radius.*

3. Using an admin account for everyday work. *A bad click while holding full power is a disaster. Use an ordinary account by default; elevate only for the task that needs it.*

4. Letting one person control a whole sensitive process. *Without separation of duties, one person can commit fraud or make an unchecked mistake. Split request from approval.*

5. Granting access and never reviewing it. *Privilege creep turns long-serving accounts into skeleton keys. Review access regularly and remove what is unused.*

6. Forgetting to remove access when people leave. *A former employee's working login is a classic breach. Remove all access the day someone leaves.*

Summary

Term	Remember
Authentication vs. authorization	Who you are vs. what you may do
Least privilege	Only the access needed, nothing more; limits damage and mistakes
Separation of duties	No one person completes a sensitive process alone; stops fraud
DAC / MAC / RBAC / ABAC	Owner decides / central policy / by role / by attributes and rules
Users, groups, roles	Manage access by job, not by person
Privilege creep	Permissions pile up over time; review and remove them

Check Your Understanding

1. In one sentence each, what do authentication and authorization answer?

2. *State the principle of least privilege, and give one reason it matters.*
3. *What is separation of duties, and what kind of harm does it prevent?*
4. *Match each model to its idea: DAC, MAC, RBAC, ABAC.*
5. *Why manage access by role instead of per person?*
6. *What is privilege creep, and name one process that fixes it.*
7. *Why is using an administrator account for everyday web browsing a bad idea?*

Answers

1. *Authentication answers "who are you?"; authorization answers "what are you allowed to do?".*
2. *Give each person, program, and device only the access it needs and no more. It matters because it limits the damage of any compromised account or mistake (and shrinks the attack surface).*
3. *No single person can complete a sensitive process alone; a critical action needs two or more people. It prevents fraud and unchecked mistakes.*
4. *DAC: the resource's owner decides. MAC: a central policy decides from labels; users can't override. RBAC: access is granted to roles, and people are given roles. ABAC: access is decided by attributes and rules.*
5. *Because you decide once what a job may do, then assign that role to people; changes are made in one place and follow people as they change jobs, which is manageable and consistent.*
6. *The slow build-up of permissions a person no longer needs. Fixes: regular access reviews (recertification), a joiner-mover-leaver process, or time-limited access.*
7. *Because if a bad click or malware strikes while you hold full administrative power, it can affect everything. Browsing as an ordinary user limits the damage of that moment.*

Practice

Think about an organization you know, or imagine a small online shop with these jobs: cashier, shift manager, owner, and a part-time bookkeeper.

1. *Write a simple **role** for each job: two or three things that role is allowed to do.*
2. *Identify one **sensitive action** (such as issuing a large refund or exporting all customer data) and apply **separation of duties**: who requests it, and who approves it?*
3. *Name one place **privilege creep** could happen, and the process you would use to catch it.*

A Worked Example

1. *Cashier: take payments, issue small refunds, look up an order. Shift manager: all of the above, approve larger refunds, open and close the till. Owner: everything, plus manage accounts. Bookkeeper: view sales reports and export accounting data, but not change prices or process orders.*
2. *Sensitive action: a refund over $50. The cashier requests it; the shift manager approves it. The person who requests is never the one who approves.*
3. *Privilege creep: the bookkeeper once covered for a manager and kept manager access. Catch it with a quarterly access review that removes any permission not matching the person's current job.*

8

Section 8

Security Controls and Defense in Depth

IN THIS SECTION

1. What a Control Is 64

2. Control Types: Preventive, Detective, Corrective 65

3. Control Categories: Technical, Administrative, Physical 65

4. Defense in Depth, Properly 66

5. Layered Defense: A Worked Example 67

6. Where Controls Fail 67

7. Security vs. Usability 68

What You Will Learn

- What a security control is.
- Control types by function: preventive, detective, and corrective.
- Control categories by nature: technical, administrative, and physical.
- Defense in depth, explained properly.
- How controls combine into layers for a real system.
- Where controls fail, and why no single one is enough.
- The trade-off between security and usability.

Before You Start

Read lessons 1 to 7. Lesson 1 introduced defense in depth; this lesson gives it structure, and organizes the many controls mentioned so far (backups, training, MFA, least privilege, logging) into a system you can reason about. No programming is needed.

---

1. What a Control Is

A **security control** is any measure you put in place to reduce risk. That is the whole definition: a control is a counter to a threat or a vulnerability. Everything you have met that protects a system is a control: a password, a backup, a locked door, a staff training session, an antivirus program, a log, a rule that says "two people must approve a payment".

Controls are how risk treatment (lesson 3) actually happens. When you decide to reduce a risk, you do it by adding controls. The art of security is choosing the right controls, enough to bring the risk down to an acceptable level, without spending more than the risk is worth or making the system unusable.

To choose well, it helps to classify controls in two ways: by **what they do** (their function), and by **what they are** (their nature). The next two topics cover each.

2. Control Types: Preventive, Detective, Corrective

Classified by **function**, when they act relative to an incident, controls fall into three main types:

Type	When it acts	What it does	Examples
Preventive	Before	Stops an incident from happening	Passwords, MFA, locks, firewalls, training, least privilege
Detective	During / after	Notices that something happened	Logs, alarms, CCTV, monitoring, intrusion detection
Corrective	After	Fixes the damage and restores normal	Backups and restore, patching, rebuilding a server, an incident response plan

A healthy security program needs all three. Preventive controls alone are not enough, because some attacks will get through; without **detective** controls you will not even know. And detection is useless if you cannot recover, which is what **corrective** controls do.

You will also hear of a few supporting types: **deterrent** controls discourage an attacker (a visible camera, a warning sign); **compensating** controls are an alternative when the ideal control is not possible (extra monitoring when you cannot yet patch a system). But the core three, prevent, detect, correct, are the ones to remember, and they map neatly onto the stages of an incident: stop it, spot it, fix it.

3. Control Categories: Technical, Administrative, Physical

Classified by **nature**, what the control actually is, there are three categories:

Category	What it is	Examples
Technical (logical)	Built into technology	Passwords, encryption, firewalls, MFA, access control, logging
Administrative (managerial)	Rules, processes, and people	Policies, training, access reviews, separation of duties, hiring checks
Physical	Things in the real world	Locks, doors, guards, cameras, a safe, a disconnected backup drive

This categorization carries a crucial lesson: **security is not only technology.** Many people think of security as firewalls and encryption, the technical controls. But administrative controls, the rules and habits, and physical controls, the locks and doors, are just as important. The

ransomware in lesson 1 got in past all the technology because of a missing administrative control: training. A server with perfect software security is still lost if someone can walk out with the physical machine.

The two classifications combine. A single control has both a type and a category: a backup is a corrective, technical control; staff training is a preventive, administrative control; a security camera is a detective (and deterrent) physical control. Thinking in this grid helps you spot gaps, for example, plenty of preventive controls but nothing to detect what gets past them.

4. Defense in Depth, Properly

Defense in depth means using multiple layers of controls, so that if one fails, others still protect the asset. *It is sometimes called a layered defense, or the "castle" approach: a moat, then walls, then a locked keep, so breaching one barrier is not enough.*

The reasoning behind it is simple and powerful: ***every single control can fail.*** *A password gets phished. A patch is missed. A firewall is misconfigured. A guard is distracted. If your security depends on one control, then the day that control fails, you are fully exposed. With layers, one failure is absorbed by the next.*

Good defense in depth deliberately mixes:

- ***Types:*** *preventive controls to stop attacks, detective controls to catch what gets through, corrective controls to recover. (Assume prevention will sometimes fail, and plan to detect and recover.)*
- ***Categories:*** *technical, administrative, and physical, so a weakness in one area is covered by another.*

Recall the attack lifecycle from lesson 5: the attacker must succeed at every stage, while the defender only has to break the chain once. Defense in depth is how you give yourself many chances to break it. It is the single most important design idea in practical security, and it is why the rest of this program teaches many controls rather than one magic solution.

5. Layered Defense: A Worked Example

Here is defense in depth for one concrete asset: a small business's customer database. Watch how the layers mix types and categories.

Layer	Control	Type	Category
1	Staff trained to spot phishing	Preventive	Administrative
2	Email filtering	Preventive	Technical
3	MFA on all accounts	Preventive	Technical
4	Least privilege: few can read the whole database	Preventive	Technical/Admin
5	The database is encrypted	Preventive	Technical
6	Logging and alerts on large data exports	Detective	Technical
7	Locked server room / secured cloud account	Preventive	Physical
8	Nightly offline backups	Corrective	Technical
9	An incident response plan	Corrective	Administrative

No single layer is perfect, and that is the point. If an attacker phishes a password (beating layer 1), MFA (layer 3) still blocks them. If they get an account anyway, least privilege (layer 4) limits what they reach, and the export alert (layer 6) catches them. If the worst happens, backups (layer 8) and the response plan (layer 9) let the business recover. An attacker now has to defeat many controls of different kinds, which is dramatically harder than beating any one.

6. Where Controls Fail

Controls are not magic, and understanding how they fail keeps you honest:

- ***Misconfiguration:*** *a control that is present but set up wrong, an "open" firewall, a backup that was never actually tested. This is one of the most common real-world failures.*
- ***Human bypass:*** *people work around controls that get in their way, propping a door open, sharing a password, clicking past a warning. A control nobody follows is no control.*
- ***Decay over time:*** *patches stop being applied, access reviews lapse, a backup drive quietly fails. Controls need maintenance.*
- ***Unknown weaknesses:*** *a brand-new attack (a zero-day) that no existing control anticipated.*

*Two habits address these. First, **test your controls**: a backup you have never restored from is a hope, not a control; a plan you have never rehearsed will not work under pressure. Second, lean again on **defense in depth**: because any one control can fail silently, you never rely on one. The*

assumption behind mature security is not "this will not fail" but "when this fails, what catches it?"

7. Security vs. Usability

*Every control has a cost, in money, in effort, and above all in **usability**. Push security too hard and you get a system so annoying that people route around it, which makes it less secure, not more. Force password changes every 30 days and people pick "Summer2026!", then "Summer2026!!". Lock screens after 30 seconds and people disable them. The most secure-looking control can backfire if it ignores how people actually work.*

*So the goal is not maximum security; it is **appropriate** security, the right controls for the risk, applied in a way people can live with. This echoes two earlier ideas: security is about **risk, not perfection** (lesson 1), and the goals **trade off** against each other and against usability (lesson 2). A good security program:*

- *Puts the strongest controls where the risk is highest, and keeps friction low elsewhere.*
- *Prefers controls that are easy to live with (a password manager, passkeys, single sign-on) over ones that fight the user.*
- *Measures whether controls are actually followed, not just whether they exist on paper.*

That balance, strong where it matters, usable everywhere, is the mark of security done well. It is also where the technical side meets the human side, which is why the next lessons turn to the rules, frameworks, and people that hold a security program together, starting with ethics and the law.

Common Mistakes

***1. Thinking security is only technology.** Administrative controls (training, policy, reviews) and physical controls (locks, doors) matter as much as technical ones. The biggest gaps are often non-technical.*

***2. Relying only on prevention.** Some attacks get through. Without detective controls you won't know, and without corrective controls you can't recover. You need all three types.*

***3. Depending on a single control.** Every control can fail. One failure should never mean full exposure. Layer them.*

***4. Never testing controls.** An untested backup or an unrehearsed plan is a hope, not a control. Test restores and rehearse responses.*

5. Letting controls decay. *Patching lapses, reviews stop, drives fail. Controls need ongoing maintenance.*

6. Over-tightening until people route around it. *A control nobody follows is no control. Match the friction to the risk, and prefer usable controls.*

Summary

Term	Remember
Control	Any measure that reduces risk
Types (function)	Preventive (before), detective (during/after), corrective (after)
Categories (nature)	Technical, administrative, physical
Defense in depth	Multiple layers, so one failure is not full exposure
Mix the layers	Different types and categories, so a gap in one is covered by another
How controls fail	Misconfiguration, human bypass, decay, unknown attacks
Security vs. usability	Aim for appropriate, usable security, not maximum

Check Your Understanding

1. What is a security control, in one sentence?
2. Name the three control types by function, and say when each acts.
3. Name the three control categories by nature, with an example of each.
4. A nightly offline backup, which type and which category is it?
5. State the core idea and the core reason behind defense in depth.
6. Give two different ways a control can fail.
7. Why can too much security make a system less secure?

Answers

1. A security control is any measure you put in place to reduce risk.
2. Preventive (acts before an incident, to stop it), detective (acts during or after, to notice it), and corrective (acts after, to fix the damage and recover).
3. Technical (built into technology, e.g. encryption), administrative (rules and processes, e.g. training), physical (real-world, e.g. a locked door).
4. A corrective control (it helps you recover after an incident) and a technical control.

5. *Idea: use multiple layers of controls so that if one fails, others still protect the asset. Reason: every single control can fail, so relying on one means full exposure the day it does.*
6. *Any two: misconfiguration (present but set up wrong), human bypass (people work around it), decay over time (maintenance lapses), or an unknown new attack.*
7. *Because controls that are too annoying make people route around them (weak predictable passwords, disabled lock screens), which leaves the system less secure than a usable control would.*

Practice

Choose one asset you care about: your laptop, your phone, a small business's customer list, or a website.

1. *List **four controls** protecting it (or that should), and label each by **type** (preventive / detective / corrective) and **category** (technical / administrative / physical).*
2. *Check your list for gaps: do you have at least one **detective** control and one **corrective** control, not only preventive ones?*
3. *Pick one control you listed and say how you would **test** that it actually works.*

A Worked Example

For a laptop with important files:

1. *Controls: a login password (preventive, technical); disk encryption (preventive, technical); keeping it in a locked drawer when out (preventive, physical); an automatic cloud backup (corrective, technical).*
2. *Gap check: plenty of prevention and a corrective backup, but no detective control. I'd add "find my device" / login alerts to notice theft or unusual sign-ins.*
3. *Testing the backup: restore a few files onto a different machine to confirm the backup is real and complete, not just that it "runs".*

9

Section 9

Ethics, the Law, and Permission to Test

IN THIS SECTION

1. Why Ethics Comes First 72

2. The One Rule, Restated 72

3. Authorization, Scope, and Rules of Engagement 73

4. Bug Bounties and Responsible Disclosure 74

5. The Shape of Computer-Crime Law 74

6. Data Protection and Privacy Duties 75

7. Professional Ethics and What to Do If You Find Something 75

What You Will Learn

- *Why the ethics rule matters more than any technique.*
- *The one rule, restated and made precise.*
- *What proper authorization looks like: scope and rules of engagement.*
- *Bug bounties and responsible disclosure: the legal way to help.*
- *The shape of computer-crime law, and how to find your own country's.*
- *Your duties around personal data and privacy.*
- *Professional ethics, and what to do if you find a vulnerability.*

Before You Start

Read lessons 1 to 8. You now understand attacks and defenses well enough that the question "what am I allowed to do with this knowledge?" is urgent. This lesson answers it.

***A note on the law:** this lesson teaches the shape of the rules, not legal advice. Laws differ by country and change over time. Where your actions might have legal consequences, confirm the current law in your own country, or ask a qualified lawyer. The specifics here are illustrative.*

---

1. Why Ethics Comes First

*By now you could describe how phishing works, how passwords are cracked, how attacks move through a network. That knowledge is exactly what a defender needs, and exactly what a criminal uses. The only thing that separates the two is **permission and intent**. That is why ethics is not a soft add-on to security; it is the line that decides whether you are a professional or a criminal, and the skills on both sides are identical.*

*This matters practically, not just morally. A security career is built on **trust**. Employers, clients, and the public hand security people deep access and powerful tools. One serious ethical breach, testing without permission, peeking at data you were not authorized to see, ends that trust and often a career, regardless of how skilled you are. In this field, your reputation for integrity is worth as much as your technical ability.*

2. The One Rule, Restated

Here is the rule from lesson 1, now stated precisely:

> ***Only ever access or test a system when you own it, or you have clear, specific, written permission from someone with the authority to grant it.***

Every word earns its place:

- ***Own it, or have permission:*** *your own lab and your own accounts are always fine. Anything else needs permission.*
- ***Clear and specific:*** *"you can test our stuff" is not enough. Permission must say which systems, what you may do, and when.*
- ***Written:*** *verbal permission protects no one. If it is not in writing, it did not happen.*
- ***From someone with the authority:*** *a friend who works at a company cannot authorize you to test the company's systems. Only the owner, or someone the owner empowered, can.*

And the rule's hard edges, repeated because they are where people go wrong:

- *Curiosity is not permission.*
- *"It was easy / exposed / public" is not permission.*
- *"I didn't change anything, I only looked" is not permission, looking is access.*
- *"I wanted to help" is not permission, unless you were invited (topic 4).*

There is no fifth exception. When in doubt, you do not have permission.

3. Authorization, Scope, and Rules of Engagement

When security work is authorized, for example a professional penetration test, the permission is written down in detail before any testing begins. Two documents matter:

- ***Scope*** *defines exactly what may be tested: which systems, addresses, applications, and accounts are* ***in scope****, and, just as importantly, what is* ***out of scope*** *and must not be touched. Testing something out of scope, even by accident, can be illegal and is always a serious breach.*
- ***Rules of engagement (RoE)*** *define how the test runs: the time window (often outside business hours), which techniques are allowed (for example, "no denial-of-service", "no social engineering of staff"), who to contact in an emergency, and how to handle any real data found.*

Underpinning both is a signed ***authorization****, sometimes called a "get out of jail free" letter, from someone with authority, stating that the tester is permitted to do this work. A professional never starts without it. These documents protect everyone: the client knows what will happen, and the tester has clear, provable permission. You will use real scope and RoE documents in the penetration testing courses.*

4. Bug Bounties and Responsible Disclosure

What if you are not hired, but you genuinely want to help, or you stumble on a flaw? There are legitimate paths, and illegitimate ones.

- ***Bug bounty programs** are standing invitations: a company publishes a policy saying "you may test these systems, within these rules, and we will reward valid findings". The published scope and rules are your written permission, as long as you stay strictly inside them. Platforms host these programs, and they are a popular, legal way to practise on real systems and earn money. The Careers course covers them.*
- ***Responsible (coordinated) disclosure** is the path when there is no bounty program and you have found a real flaw, usually in something you were legitimately using. You report it privately to the owner, give them reasonable time to fix it, and do not publish details or exploit it in the meantime. Done well, this helps everyone and keeps you on the right side of the line.*

*The crucial boundary: a bug bounty's published scope is permission; finding a flaw by chance is **not** permission to go digging further. The moment you start probing beyond what you were invited to do, or beyond what you encountered in normal use, you have crossed into unauthorized access.*

5. The Shape of Computer-Crime Law

Almost every country has laws against unauthorized access to computer systems. The details vary, but the shape is consistent:

- ***Unauthorized access** (getting into a system you are not permitted to) is an offense, often regardless of whether you caused any harm. Simply logging in without permission can be a crime.*
- ***Unauthorized modification** (changing data or systems) is treated more seriously.*
- ***Causing damage or loss**, or acting for financial gain, raises the penalties further.*
- ***Making or supplying attack tools** for criminal use can itself be an offense.*

*Well-known examples include the United States' Computer Fraud and Abuse Act and the United Kingdom's Computer Misuse Act; the European Union and most countries have equivalents. **Cambodia** has its own legal framework governing computer and online offenses, and it continues to develop. Because specifics and penalties change, **confirm the current Cambodian law before relying on any detail**, and treat the safe rule, no access without written permission, as your guide everywhere. Ignorance of the law is not a defense, and "I'm a student" or "I was learning" does not make unauthorized access legal.*

6. Data Protection and Privacy Duties

Security people handle personal data constantly, logs, user records, the contents of a breach, and that carries legal and ethical duties. Personal data means information about identifiable people: names, phone numbers, addresses, health, finances, messages.

The widely shared principles (seen in the EU's GDPR and many national laws, including developing frameworks in Cambodia) include:

- ***Collect only what you need,** and use it only for the stated purpose.*
- ***Protect it** with appropriate security (the whole point of this field).*
- ***Limit who can see it** (least privilege, lesson 7).*
- ***Keep it only as long as needed,** then delete it.*
- ***Respect people's rights** over their own data.*
- ***Report breaches** to authorities and affected people, often within a legal deadline.*

For you as a learner and future professional, the practical duty is simple: when your work exposes you to real personal data, treat it with care, access only what you need, never copy or share it beyond the task, and never use it for curiosity or personal gain. Mishandling personal data can be both a crime and a firing offense.

7. Professional Ethics and What to Do If You Find Something

*Beyond the law, the security profession holds itself to **codes of conduct**. Certifications and professional bodies ask members to act honestly, protect the public and their clients, work only within their competence and authorization, and disclose findings responsibly. The common thread is the one you already know: use your skills to protect, act only where permitted, and be honest.*

*Finally, a situation you will meet: **you find a vulnerability.** What do you do?*

1. ***Stop.** Do not probe further, and do not exploit it. Note what you already saw in normal use.*
2. ***Check for a path.** Does the owner have a bug bounty or a security contact (often a `security.txt` file or a "report a vulnerability" page)?*
3. ***Report privately** to the owner, with enough detail to reproduce it. Do not post it publicly.*
4. ***Give them reasonable time** to fix it before any disclosure, and keep records of your good-faith contact.*
5. ***Do not demand payment.** Asking for money to stay quiet looks like extortion, which is a crime, even if you meant well.*

Report it the way you would want someone to report a flaw in your system: privately, helpfully, and without making things worse. That instinct, defend, don't exploit; help, don't harm; act only with permission, is the ethical core of everything that follows in this program. With it firmly in place, the next lessons look at the frameworks and roles that organize security work at scale.

Common Mistakes

1. Treating ethics as optional. *Permission and intent are the only things separating a professional from a criminal; the skills are identical.*

2. Relying on verbal or vague permission. *Permission must be written, specific about systems and actions, and from someone with authority. Otherwise you have none.*

3. Thinking "I only looked" is safe. *Looking is access. Unauthorized access is often a crime regardless of harm.*

4. Treating a found bug as a licence to dig. *Encountering a flaw in normal use is not permission to probe further. That step crosses into unauthorized access.*

5. Assuming student status or good intentions protect you. *The law does not exempt learners or the well-meaning. Only your own systems or written permission do.*

6. Mishandling personal data. *Access only what you need, never copy or share it, never use it for curiosity. It can be both a crime and a firing offense.*

7. Asking for payment to stay quiet. *Demanding money over a vulnerability looks like extortion. Report responsibly; let any reward come through an official bounty.*

Summary

Idea	Remember
Why ethics first	Permission and intent are the only line between professional and criminal
The one rule	Access/test only what you own, or have clear, specific, written permission for
Scope & RoE	What may be tested, and how; signed authorization before any testing
Bug bounty	A published invitation = permission, within its rules
Responsible disclosure	Report flaws privately; give time to fix; don't exploit or publish
Computer-crime law	Unauthorized access is an offense, often regardless of harm; confirm local law
Personal data	Collect little, protect it, limit access, don't misuse it
If you find a bug	Stop, report privately, give time, don't demand payment

Check Your Understanding

1. What two things separate a security professional from a criminal, given identical skills?
2. State the one rule precisely, and explain why "written" and "from someone with authority" matter.
3. What is the difference between scope and rules of engagement?
4. How can testing a real company's systems ever be legal without being hired by them?
5. Is "I only looked, I didn't change anything" a legal defense? Why or why not?
6. Name three duties you have when handling personal data.
7. You find a serious flaw in a website you use. List the first three things you should do.

Answers

1. Permission and intent. The technical knowledge is the same on both sides.
2. Only access or test a system you own, or have clear, specific, written permission for, from someone with authority. Written, because verbal permission protects no one and can't be proven; from someone with authority, because only the owner (or their delegate) can actually grant it.
3. Scope defines what may be tested (which systems are in or out of bounds); rules of engagement define how (timing, allowed techniques, contacts, handling of data).
4. Through a bug bounty program, whose published scope and rules are a standing written permission, as long as you stay strictly within them.

5. *No. Looking is access, and unauthorized access is often a crime regardless of whether you changed anything or caused harm.*
6. *Any three: collect only what's needed; use it only for its purpose; protect it; limit who can see it; keep it only as long as needed; respect people's rights; report breaches.*
7. *Stop and don't probe further or exploit it; check for a bug bounty or security contact; report it privately to the owner with enough detail to reproduce. (Then give time to fix, and never demand payment.)*

Practice

1. *Rewrite the one rule in your own words, in a single sentence you could explain to a friend.*
2. *A friend says: "This local shop's website looks really insecure. Let me just try a few things to see if I can get in, I'd tell them afterwards so it's fine." Give two reasons this is not fine, and one legal way they could help instead.*
3. *Find out whether a website or app you use has a way to report security issues (look for a "security" page, a `security.txt`, or a vulnerability-disclosure policy). Write down what you found, or note that you could not find one.*

A Worked Example

1. *"Don't touch any computer or account that isn't mine unless the owner has clearly, and in writing, said I can, and said exactly what I'm allowed to do."*
2. *Not fine because: trying to get in is unauthorized access, which is a crime even if nothing is changed and even if they'd tell the shop afterwards; and "I'd tell them after" is not the owner's permission in advance. A legal way to help: contact the shop privately, explain politely that they may have a security issue, and offer to help only if they ask, through responsible disclosure, not by breaking in first.*
3. *(Your findings. For example: "Checked a bank app's website; it has a 'Report a vulnerability' page with an email address and asks for private disclosure, that's a responsible-disclosure path.")*

10

Section 10

Security Frameworks and Standards

IN THIS SECTION

1. Why Frameworks Exist 80

2. The NIST Cybersecurity Framework 2.0 81

3. The CIS Critical Security Controls 81

4. ISO/IEC 27001 and 27002 82

5. MITRE ATT&CK 82

6. Compliance vs. Security 83

7. Choosing Where to Start 83

What You Will Learn

- *Why security frameworks exist, and what they give you.*
- *The NIST Cybersecurity Framework 2.0, and its six functions.*
- *The CIS Critical Security Controls, a practical starting list.*
- *ISO/IEC 27001 and 27002, the international standard.*
- *MITRE ATT&CK, the shared language of attacker behavior.*
- *The difference between compliance and real security.*
- *How a small business can choose where to start.*

Before You Start

Read lessons 1 to 9. You now know the ideas (goals, risk, controls, ethics); this lesson shows the organized, published ways that professionals apply them, so you do not have to invent security from scratch. No programming is needed.

---

1. Why Frameworks Exist

A security program has dozens of moving parts: assets, risks, controls, people, processes. Left to invent it alone, every organization would miss something, and you could never compare one organization's security to another's. ***Frameworks*** *solve this. A security framework is a published, structured set of guidance, built by experts and refined over years, that tells you what to do and often how to organize it.*

Frameworks give you four things:

- ***Completeness:*** *a checklist so you do not forget whole areas (like "we had firewalls but no backups").*
- ***A common language:*** *so your team, your auditors, and your partners mean the same things.*
- ***Credibility:*** *following a recognized framework shows customers and regulators you take security seriously.*
- ***A starting point:*** *you adapt proven guidance instead of starting from a blank page.*

There are many frameworks, and they overlap; you do not use all of them. Some describe how to run a whole program (NIST CSF, ISO 27001), some are a prioritized list of controls (CIS), and one is a catalogue of attacker behavior (MITRE ATT&CK). This lesson introduces the main ones so you recognize them; the GRC course goes deep.

2. The NIST Cybersecurity Framework 2.0

*The **NIST Cybersecurity Framework** (CSF) is one of the most widely used. It was created by the United States' National Institute of Standards and Technology, is free, and is used worldwide. Its strength is a simple, memorable structure built around **six functions** that describe everything a security program does:*

Function	The question it answers
Govern	Who is responsible, and what are our risks, roles, and policies?
Identify	What do we have, and what matters? (assets and risks)
Protect	How do we safeguard it? (controls, access, training)
Detect	How do we notice when something happens? (monitoring, logs)
Respond	What do we do during an incident? (containment, communication)
Recover	How do we return to normal? (backups, lessons learned)

*Notice how these map onto ideas you already know: Identify is assets and risk (lessons 1–3), Protect is controls (lessons 6–8), Detect/Respond/Recover are the defensive courses later in the program. **Govern** was added in version 2.0 (2024) to stress that security is a leadership responsibility, not just a technical one. The six functions are a superb mental model: for any organization, you can ask "how are they doing on each of the six?" and quickly find the gaps.*

3. The CIS Critical Security Controls

*Where the NIST CSF describes what areas to cover, the **CIS Critical Security Controls** (from the Center for Internet Security) give a concrete, **prioritized list of specific things to do**, ordered so that the most effective come first. For someone asking "but what do I actually do on Monday?", CIS is the most practical answer.*

The current version has 18 controls, grouped so that a small organization can start with the first, highest-impact ones, such as:

- *Know your hardware and software (you cannot protect what you do not know you have).*
- *Protect data, and configure devices securely.*
- *Control accounts and access, and use MFA.*
- *Manage vulnerabilities (patching).*
- *Keep and monitor logs.*
- *Run awareness training.*
- *Have a backup and recovery capability.*

*CIS also defines **Implementation Groups**, tiers that tell a small business (IG1), a mid-sized one (IG2), or a large one (IG3) how far to go. This makes it approachable: a small shop implements IG1 and already blocks the large majority of common attacks, which echoes the lesson from earlier, ordinary, well-applied controls stop most attacks.*

4. ISO/IEC 27001 and 27002

***ISO/IEC 27001** is the leading international standard for information security management. Where NIST CSF is a flexible framework, ISO 27001 is a formal **standard you can be certified against**: an independent auditor checks your security management and, if it meets the requirements, issues a certificate recognized worldwide.*

*Its central idea is an **Information Security Management System (ISMS)**: not a product, but an ongoing, documented process for managing security based on risk, deciding what to protect, applying controls, checking they work, and improving, continuously. Its companion document, **ISO/IEC 27002**, is a detailed catalogue of controls to choose from.*

Why it matters: many larger companies, and many contracts, require their suppliers to be ISO 27001 certified. For a business that sells to such customers, certification is often a condition of doing business, not just a nice-to-have. It is more formal and more effort than the CIS controls, and it is usually a goal for a maturing organization rather than a first step.

5. MITRE ATT&CK

***MITRE ATT&CK** is different from the others: it is not a program framework or a control list, but a **detailed, public knowledge base of how attackers actually behave**, organized by the stages of an attack (reconnaissance, initial access, privilege escalation, lateral movement, and so on, the attack lifecycle from lesson 5).*

*For each stage it catalogues the specific **techniques** real attackers use, with descriptions, real-world examples, and, crucially, how to **detect and defend** against each. It is maintained from observation of actual attacks, so it reflects the real world, not theory.*

Why it is so valuable, especially for defenders:

- *It gives everyone a **common language** for attacker behavior ("we saw technique T-such-and-such") so teams and tools can communicate precisely.*
- *Defenders use it to **measure coverage**: which attacker techniques can we detect, and where are our blind spots?*
- *It connects attack to defense: for every technique an attacker might use, it suggests what to watch for.*

You will use ATT&CK heavily in the defensive courses (Security Operations, Detection Engineering, Threat Intelligence) and in red teaming. For now, know that when security people talk about "ATT&CK", they mean this shared map of how attacks work and how to catch them.

6. Compliance vs. Security

Here is one of the most important distinctions in the whole field. ***Compliance*** *means meeting the formal requirements of a framework, standard, or law, often to pass an audit.* ***Security*** *means actually being hard to attack. They overlap, but they are* ***not the same thing****, and confusing them is dangerous.*

- *You can be* ***compliant but insecure:*** *you ticked every box for the audit, but the controls are weak in practice, out of date, or ignored day to day. "We passed the audit" is not "we are safe". Many breached companies were certified at the time.*
- *You can be* ***secure but not (yet) compliant:*** *a small team with excellent, well-practised security habits that has not done the paperwork for a formal certificate.*

The right way to see it: ***compliance is a floor, not a ceiling.*** *Frameworks and standards are a valuable baseline and a way to prove diligence, but the goal is real security, not just a passing grade. A good professional uses frameworks to be thorough and to communicate, while never mistaking a certificate for safety. Regulations you may meet, such as* ***PCI DSS*** *(for handling card payments) or data-protection laws, are compliance requirements; treat them as a minimum you build real security on top of.*

7. Choosing Where to Start

With several frameworks, which do you use? They are complementary, not competing, and the answer depends on the organization.

- ***A small business asking "what do we do first?"*** *→ start with the* ***CIS Controls, Implementation Group 1****. It is concrete, prioritized, and blocks most common attacks without overwhelming a small team.*
- ***An organization wanting to structure a whole program and talk to leadership*** *→ use the* ***NIST CSF*** *and its six functions as the backbone, then fill it in with specific controls (often CIS).*
- ***A company that must prove its security to big customers or regulators*** *→ work toward* ***ISO 27001 certification****.*
- ***A team building detection and defense*** *→ use* ***MITRE ATT&CK*** *to guide and measure what they can catch.*

A common, sensible combination is: NIST CSF to organize the program, CIS controls to decide what to implement first, MITRE ATT&CK to guide detection, and ISO 27001 later if the business needs the certificate. You do not need to master these now. The goal of this lesson is that when you hear "we follow NIST CSF" or "they're ISO 27001 certified" or "that maps to ATT&CK", you know what each means and why it is there. The next lesson looks at the people who do this work: the roles and careers in security.

Common Mistakes

1. Trying to use every framework at once. *They serve different purposes and overlap. Pick what fits the need: a program model, a control list, a standard, or an attack map.*

2. Treating a framework as a magic checklist. *Frameworks guide thoroughness; they do not implement themselves, and ticking boxes is not the same as being secure.*

3. Confusing compliance with security. *Passing an audit is a floor, not a ceiling. Many breached companies were "compliant" at the time.*

4. Thinking standards are only for big companies. *CIS Implementation Group 1 is designed for small organizations and blocks most common attacks.*

5. Ignoring the Govern function. *Security is a leadership responsibility, not just a technical one. NIST CSF 2.0 added Govern to make this explicit.*

6. Mistaking MITRE ATT&CK for a program framework. *It is a knowledge base of attacker behavior for detection and defense, not a plan for running a whole security program.*

Summary

Framework	What it is	Best for
NIST CSF 2.0	Six functions: Govern, Identify, Protect, Detect, Respond, Recover	Structuring a whole program; talking to leadership
CIS Controls	A prioritized list of specific controls, with tiers (IG1–IG3)	Knowing what to do first
ISO/IEC 27001 / 27002	International standard you can be certified against; an ISMS	Proving security to customers and regulators
MITRE ATT&CK	Knowledge base of attacker techniques, with detection	Building and measuring detection and defense
Compliance vs. security	Meeting requirements vs. actually being hard to attack	Remember: compliance is a floor, not a ceiling

Check Your Understanding

1. Give two things a framework provides that inventing your own does not.
2. Name the six functions of the NIST CSF 2.0.
3. How do the CIS Controls differ in purpose from the NIST CSF?
4. What is an ISMS, and which standard is built around it?
5. What is MITRE ATT&CK, and who finds it most useful?
6. Explain "compliance is a floor, not a ceiling" in your own words.
7. A small shop asks where to begin. What do you recommend, and why?

Answers

1. Any two: completeness (a checklist so you don't miss areas); a common language; credibility with customers and regulators; a proven starting point.
2. Govern, Identify, Protect, Detect, Respond, Recover.
3. NIST CSF describes what areas a program should cover (a structure); the CIS Controls give a concrete, prioritized list of specific things to do, ordered by impact.
4. An Information Security Management System: an ongoing, documented process for managing security based on risk. ISO/IEC 27001 is built around it.
5. A public knowledge base of how attackers actually behave, organized by attack stage, with detection and defense guidance. Defenders (SOC, detection, threat intelligence) and red teams find it most useful.

6. *Meeting the formal requirements (passing an audit) is a minimum baseline, not the goal. You can be compliant yet insecure, so real security must be built on top of compliance, not mistaken for it.*
7. *Start with the CIS Controls, Implementation Group 1: it's concrete, prioritized, sized for a small team, and blocks the majority of common attacks.*

Practice

1. *Take any small organization (real or imagined) and write one sentence for each of the **six NIST CSF functions**, saying what it would mean for them (e.g. "Detect: we review login alerts each morning").*
2. *From the CIS-style starter list in topic 3, pick the **three controls** you think would most reduce risk for that organization, and say why.*
3. *In one or two sentences, describe a situation where an organization could be **compliant but insecure**.*

A Worked Example

For a small online shop:

1. *Govern: the owner sets a simple security policy and reviews risks each quarter. Identify: list every device, account, and the customer data we hold. Protect: MFA on all accounts, least privilege, encrypted database. Detect: alerts on large data exports and failed logins. Respond: a one-page plan naming who does what in an incident. Recover: nightly offline backups we test monthly.*
2. *Top three controls: MFA on all accounts (defeats stolen passwords), nightly tested backups (defeats ransomware), and patching/updates (closes the holes worms and exploits use). These block the most common serious attacks.*
3. *Compliant but insecure: the shop passed a card-payment audit a year ago, but since then it stopped applying updates and never tested its backups, so it holds a valid certificate while being wide open to ransomware.*

11

Section 11

Security Roles and the Industry

IN THIS SECTION

1. The Shape of the Industry 88

2. Offensive Roles 89

3. Defensive Roles 89

4. Engineering and Architecture Roles 90

5. Governance, Risk, and Compliance Roles 90

6. How the Roles Work Together 91

7. Certifications at a Glance 92

What You Will Learn

- *The overall shape of the security industry.*
- *Offensive roles: finding weaknesses by attacking (with permission).*
- *Defensive roles: watching, catching, and recovering from attacks.*
- *Engineering and architecture roles: building security in.*
- *Governance, risk, and compliance roles: the management side.*
- *How the roles work together, including the purple team.*
- *The main certifications, at a glance.*

Before You Start

Read lessons 1 to 10. You now understand what security work involves; this lesson maps that work onto real jobs, so you can see where you might fit, and how the courses ahead lead to each. No programming is needed.

---

1. The Shape of the Industry

Security is not one job; it is a whole industry of specialties, the same way "medicine" includes surgeons, nurses, radiologists, and administrators. People often sort security roles into a few broad families:

- ***Offensive** ("red"): find weaknesses by attacking systems, with permission.*
- ***Defensive** ("blue"): watch for, detect, and respond to attacks.*
- ***Engineering / architecture**: build and run systems so they are secure by design.*
- ***Governance, risk, and compliance (GRC)**: manage security as a business function, policy, risk, and audits.*

*These families map directly onto the paths in this program (Offensive, Defensive, Engineering, and the Specialized/GRC courses). Few people do all of them; most specialize, while keeping a working understanding of the others. Importantly, you do **not** have to be a brilliant hacker to have a security career. Many of the best-paid, most in-demand roles are defensive, engineering, or GRC, and value care, communication, and persistence as much as raw technical flair.*

A note for Cambodia and the region: demand for security people far outstrips supply almost everywhere, including locally and for remote work with foreign companies. This is a field where skills, a portfolio, and persistence open doors, which the Careers course covers in depth.

2. Offensive Roles

*Offensive security professionals attack systems, **with permission**, to find weaknesses before criminals do. They are sometimes called **ethical hackers**. The ethics rule from lesson 9 is their daily reality: everything is scoped and authorized.*

Common offensive roles:

- ***Penetration tester (pentester):** hired to test a specific system (a web app, a network) within a defined scope and time, then report the weaknesses and how to fix them.*
- ***Red teamer:** runs broader, goal-based, stealthy engagements that emulate a real attacker over weeks, to test not just systems but the organization's detection and response.*
- ***Vulnerability researcher / exploit developer:** digs deep into software to find new, unknown weaknesses (and often reports them responsibly).*
- ***Bug bounty hunter:** works independently, testing companies' systems through their published bounty programs, often for reward.*

*What the job is really like: a lot of methodical testing, research, and, crucially, **report writing**, explaining clearly what is wrong and how to fix it. The thrill of "breaking in" is a small part; the value is in the clear, actionable report. This family is the Offensive path: Web Application Security, Network Penetration Testing, Reverse Engineering, Exploitation, Red Teaming, and more.*

3. Defensive Roles

*Defensive security, the **blue team**, is the larger side of the industry by number of jobs, and where many people start. Defenders keep watch, catch attacks, and lead the recovery.*

Common defensive roles:

- ***SOC analyst:** works in a Security Operations Center, watching alerts and logs, triaging what is real, and raising incidents. A very common entry point into security.*
- ***Incident responder:** takes over when something serious happens, contains it, removes the attacker, and restores normal operations.*
- ***Digital forensics analyst:** investigates what happened, recovering and analyzing evidence from disks, memory, and networks.*
- ***Threat intelligence analyst:** studies attackers and their methods to help the organization anticipate and prioritize.*
- ***Threat hunter / detection engineer:** proactively searches for attackers that alerts missed, and writes better detections.*

Defensive work rewards curiosity, attention to detail, calmness under pressure, and good communication. It is less about dramatic hacking and more about noticing the one wrong thing among thousands of normal events. This family is the Defensive path: Security Operations, Incident Response, Digital Forensics, Malware Analysis, Threat Intelligence, and the Dark Web course.

4. Engineering and Architecture Roles

*While red and blue teams test and watch, **security engineers and architects build security in** from the start, which is cheaper and more effective than bolting it on later.*

Common roles:

- ***Security engineer:** builds and runs security tools and defenses, firewalls, monitoring, identity systems, and automates security tasks.*
- ***Application / product security engineer:** works with developers to build software securely, reviews code and designs, and runs security testing (the Secure Development course).*
- ***Cloud security engineer:** secures cloud environments (AWS, Azure, Google Cloud), a fast-growing, well-paid specialty.*
- ***Security architect:** designs how an organization's whole security fits together, choosing the controls and how systems connect safely.*

*These roles suit people who like **building** and often have a development or systems background. They are among the most in-demand and best-paid in the field. This family is the Engineering path: Secure Development, Cloud Security, Container Security, and Identity and Access Management.*

5. Governance, Risk, and Compliance Roles

*Not all security work is technical. **GRC** roles manage security as a business function, and are essential to any mature organization.*

Common roles:

- ***Security analyst / GRC analyst:** manages policies, risk assessments (lesson 3), and compliance with frameworks (lesson 10).*
- ***Auditor:** independently checks whether controls are in place and working.*
- ***Compliance manager:** ensures the organization meets its legal and contractual obligations (data protection, PCI DSS, ISO 27001).*

- ***CISO (Chief Information Security Officer):*** *the senior leader responsible for the whole security program, bridging security and business leadership.*

GRC roles reward people who are organized, clear communicators, and good with people and process. They are an excellent path for those who find the strategy, risk, and people side more interesting than deep technical work, and they often pay very well. This family is covered by the GRC course.

6. How the Roles Work Together

These are not rival camps; they form one system of defense, and the best security happens when they cooperate.

- ***Red and blue together make purple.*** *A* ***purple team*** *is not a permanent role but a way of working: red (attack) and blue (defend) collaborate directly, with red showing blue exactly how an attack works so blue can detect and block it. Instead of red "winning" by breaking in, both sides improve the organization's defenses together. This is the most productive use of offensive skills.*
- ***Engineers build what blue monitors and red tests.*** *A finding from a pentest becomes a fix by an engineer and a new detection by the SOC.*
- ***GRC sets the priorities*** *that direct everyone, based on risk, and reports the results to leadership.*

Understanding every role, even the ones you do not do, makes you better at your own, because you know how your work fits the whole. A pentester who understands the SOC writes more useful reports; an engineer who understands GRC builds what the business actually needs. This is why a program like this one teaches all the families before you specialize.

7. Certifications at a Glance

Security uses **certifications** heavily, as a common (if imperfect) way to signal knowledge to employers. You do not need any to start learning, and this program is not built around exams, but you should recognize the main names. The Careers course covers how and whether to pursue them.

Level	Example certifications	For
Entry / foundational	CompTIA Security+, CompTIA Network+	Proving broad fundamentals; many first jobs ask for Security+
Offensive	OSCP, CEH, and others	Pentesting and red-team roles; OSCP is hands-on and well-respected
Defensive / operations	Blue-team and SOC certifications	SOC, incident response, forensics
Management / GRC	CISSP, CISM, ISO 27001 Lead Auditor	Senior, architecture, and governance roles
Cloud	Cloud-provider security certifications	Cloud security roles

Two honest points. First, **certifications help you get interviews, but skills and a portfolio get you hired** (and keep you employed). Hands-on practice, labs, CTFs, and real projects, matters more than collecting letters. Second, do not feel you must chase certifications to belong in security; many excellent practitioners have few, and build their reputation on demonstrated work.

That demonstrated work is exactly what the rest of this program builds toward. You have now seen what security is, how attacks and defenses work, the rules that govern them, the frameworks that organize them, and the people who do the work. The final lesson turns all of this into a plan: your roadmap through the program, and your first steps into the field.

Common Mistakes

1. Thinking you must be an elite hacker to work in security. Most roles are defensive, engineering, or GRC, and value care, communication, and persistence as much as raw offensive skill.

2. Picturing the job as constant dramatic hacking. Offensive work is mostly methodical testing and report writing; defensive work is noticing the one wrong thing among thousands of normal events.

3. Seeing red and blue as enemies. *They are one system. Purple teaming, where they collaborate, produces the best defenses.*

4. Believing certifications equal skill. *Certifications open interviews; demonstrated skills and a portfolio get and keep the job.*

5. Overlooking GRC and engineering. *The non-"hacking" roles are among the most in-demand and best-paid, and suit many people better.*

6. Waiting to feel "qualified" before starting. *Demand far exceeds supply. Skills, a portfolio, and persistence open doors, especially for remote work.*

Summary

Family	Roles	Program path
Offensive (red)	Pentester, red teamer, researcher, bounty hunter	Offensive courses
Defensive (blue)	SOC analyst, incident responder, forensics, threat intel	Defensive courses
Engineering / architecture	Security / appsec / cloud engineer, architect	Engineering courses
GRC	GRC analyst, auditor, compliance manager, CISO	GRC course
Purple team	Red + blue collaborating to improve defenses	Red Teaming course
Certifications	Security+ (entry), OSCP (offensive), CISSP (management), cloud	Careers course

Check Your Understanding

1. Name the four broad families of security roles.
2. Why doesn't a security career require you to be a brilliant hacker?
3. What does a SOC analyst do, and why is it a common first job?
4. What is the difference between a penetration tester and a red teamer?
5. What is a purple team, and why is it productive?
6. What do engineering/architecture roles add that red and blue teams do not?
7. In one sentence, what is the honest relationship between certifications and getting hired?

Answers

1. Offensive (red), defensive (blue), engineering/architecture, and governance-risk-compliance (GRC).
2. Because most roles are defensive, engineering, or GRC, which value attention to detail, communication, persistence, and process at least as much as offensive skill.
3. A SOC analyst watches alerts and logs, triages what's real, and raises incidents. It's a common entry point because it builds broad exposure and is in high demand.
4. A pentester tests a specific system within a defined scope and time and reports the weaknesses; a red teamer runs broader, stealthy, goal-based engagements that also test the organization's detection and response.
5. A way of working where red (attack) and blue (defend) collaborate directly, with red showing blue how an attack works so blue can detect and block it. It's productive because both sides improve the defenses together, rather than red just "winning".
6. They build security in from the start, the tools, systems, and designs, which is cheaper and more effective than bolting it on, and which red tests and blue monitors.
7. Certifications help you get interviews, but skills and a portfolio are what actually get and keep you hired.

Practice

1. Of the four role families, which **one** appeals to you most right now, and why? (There is no wrong answer; it may change.)
2. Pick one specific role from this lesson and write two or three sentences describing what a typical day might involve, in your own words.
3. Look back at the program's paths (see the repository's main page or lesson 1). Which **path** would move you toward the role you chose, and which course in it are you most curious about?

A Worked Example

1. Defensive appeals most: I like the idea of watching for and catching attackers, and it's a strong way to enter the field.
2. SOC analyst: they start a shift by reviewing overnight alerts, deciding which are false alarms and which are real, investigating the suspicious ones by checking logs, and escalating anything serious to an incident responder, all while keeping clear notes.
3. The Defensive path would move me toward it, and I'm most curious about the Security Operations course, with Digital Forensics a close second.

12

Section 12

Your Learning Roadmap

IN THIS SECTION

1. How This Program Is Organized 96

2. The Foundations, and Why They Come First 97

3. Choosing a Path 97

4. Building Skills in the Home Lab 98

5. Practising: CTFs and Challenges 98

6. Building a Portfolio 99

7. Staying Current, and Staying Safe 99

8. Your Next Step 100

What You Will Learn

- *How this whole program is organized, and why.*
- *The Foundations, and why everyone does them first.*
- *How to choose a path once Foundations are done.*
- *How to build real skill in a home lab.*
- *How to practise with CTFs and challenges.*
- *How to build a portfolio that gets you hired.*
- *How to stay current, and stay safe and ethical, for the long run.*
- *Your concrete next step.*

Before You Start

Read lessons 1 to 11. This is the final lesson of Security Fundamentals. It does not add new security theory; it turns everything you have learned into a practical plan for the months and years ahead. No programming is needed.

---

1. How This Program Is Organized

The program is built as a path from knowing nothing to being ready for professional work. It has five stages, in a deliberate order:

Stage	Courses	Purpose
Foundations	Security basics, how computers work, Linux, networking, Windows, scripting, Python, a home lab, cryptography	The ground every security person stands on
Offensive	Web, mobile, network, wireless, reverse engineering, exploitation, red teaming	How attacks really work, hands-on
Defensive	Security operations, incident response, forensics, malware analysis, threat intelligence, the dark web	How to watch, catch, and recover
Engineering	Secure development, cloud, containers, identity	How to build systems that resist attack
Specialized	Industrial systems, fraud and banking, blockchain, AI security, GRC, careers, and a final capstone	Deep fields, and turning skills into a job

The order is not arbitrary. Each stage depends on the ones before it. You cannot meaningfully attack or defend a network you do not understand, and you cannot understand a network without the Foundations. So the single most important rule of the roadmap is: ***do the Foundations first, in order, and do not rush them.***

2. The Foundations, and Why They Come First

It is tempting to jump straight to the exciting courses, "I want to hack things now". Resist it. Almost everyone who struggles later skipped or skimmed the Foundations, and almost everyone who progresses smoothly built them solidly.

Here is why each Foundation matters:

- ***How Computers Work:*** *you cannot analyze what a program does if you do not know what a program is.*
- ***Linux:*** *the command line is the native environment of security tools; you will live in it.*
- ***Networking:*** *almost every attack and defense involves the network; this is the language of the field.*
- ***Windows and Active Directory:*** *most organizations run on Windows; most attacks target it.*
- ***Scripting and Python:*** *automation turns slow manual work into fast, repeatable work, and lets you build your own tools.*
- ***The Home Lab:*** *your safe, legal place to practise everything (topic 4).*
- ***Cryptography:*** *the math behind passwords, HTTPS, and signatures, understood, not memorized.*

Think of the Foundations as learning the alphabet and grammar before writing essays. They are less flashy, but they are what separate someone who truly understands security from someone who can only follow recipes. Give them the time they deserve.

3. Choosing a Path

Once the Foundations are solid, you specialize. The three main paths match the role families from lesson 11:

- ***Offensive****, if you love figuring out how things break, puzzles, and the attacker's mindset.*
- ***Defensive****, if you like watching, investigating, noticing patterns, and staying calm under pressure. This path has the most jobs and is a common, reliable way in.*
- ***Engineering****, if you like building, automating, and designing systems well, often a great fit if you come from development.*

You do not have to choose forever, or choose only one. A sensible approach is to do the path that excites you most first, then broaden. Understanding more than one side makes you more valuable: defenders who understand attacks catch more; builders who understand both build better. The ***Specialized*** *courses (fraud, cloud, AI, industrial systems, GRC) layer on top once you have a path, and often follow the local job market, in Cambodia, for instance, fraud and banking security and cloud are in strong demand.*

How to decide: if unsure, start ***Defensive****. It has the broadest job market, teaches you to think about attacks from the receiving end, and gives a strong base to pivot from later.*

4. Building Skills in the Home Lab

You cannot learn security by reading alone. *Reading about lock-picking does not make you a locksmith. The single most important study habit in this whole field is* ***hands-on practice in a safe, legal environment****, and that environment is your home lab (built in the Home Lab course).*

A home lab is a set of virtual machines on your own computer: an attacker machine, some deliberately vulnerable targets, and a small network, all isolated from the real world and from systems you do not own. In it, the ethics rule is satisfied by design: everything belongs to you, so you can attack and defend freely and legally.

How to use it well:

- ***Do, don't just watch.*** *For every technique you learn, perform it yourself in the lab.*
- ***Break things and fix them.*** *Attack a target, then defend it. Watching both sides teaches far more than either alone.*
- ***Take notes as you go.*** *Your notes become your portfolio (topic 6) and your memory.*
- ***Reset and repeat.*** *Snapshots let you return to a clean state and try again.*

Every hands-on course in this program assumes you are practising in your lab, not just reading. Build it early, and use it constantly.

5. Practising: CTFs and Challenges

Beyond your own lab, the security community runs a huge amount of free, legal practice, and it is one of the best ways to sharpen skills and have fun doing it.

- ***CTFs (Capture The Flag)*** *are security competitions where you solve challenges, finding a hidden "flag" by exploiting a weakness, in categories like web, cryptography, forensics, reverse engineering, and binary exploitation. They range from beginner-friendly to very hard. The final course of this program is devoted to them.*

- ***Practice platforms*** *host deliberately vulnerable machines and guided challenges you can work through at your own pace, legally, because they exist for exactly this purpose.*
- ***Wargames and labs*** *teach specific skills step by step.*

Why practise this way: it is legal (the targets are made for it), it is graded (you know when you succeed), it builds real problem-solving stamina, and it connects you to a community. Doing challenges regularly, even 30 minutes a few times a week, compounds into serious skill over months. It also gives you concrete things to write about, which brings us to the portfolio.

6. Building a Portfolio

Remember from lesson 11: ***skills and a portfolio get you hired.*** *A portfolio is simply evidence of what you can do, and you can start building it now, long before your first job. It matters especially when you lack formal experience, because it proves ability directly.*

Good portfolio pieces:

- ***Write-ups:*** *clear explanations of how you solved a CTF challenge or a lab, what the weakness was, how you found it, and how to fix it. Write-ups show both skill and communication, which employers prize.*
- ***A blog or notes site:*** *a simple place to publish what you learn. Teaching something proves you understand it.*
- ***Projects on a code-sharing site:*** *small security tools or scripts you have written (the Python and scripting courses produce these).*
- ***Documented lab work:*** *your home-lab setups and experiments.*
- ***Bug bounty or responsible-disclosure results****, once you are ready and strictly within the rules (lesson 9).*

Two tips. Keep it ***honest*** *(never claim work you did not do; integrity is everything in this field, lesson 9) and keep it* ***clear*** *(a well-explained simple project beats a vague complex one). Start a write-up habit with your very next challenge or lab.*

7. Staying Current, and Staying Safe

Security never stands still; new attacks, tools, and defenses appear constantly. Staying effective is a career-long habit, not a one-time achievement. Ways to keep up:

- ***Follow reputable sources:*** *security news, vendor and government advisories (like CISA), and respected practitioners.*

- ***Join communities:*** *local security groups, online forums, and chats. The field is unusually generous with knowledge, and in Cambodia a growing local community and remote communities both help.*
- ***Keep practising:*** *ongoing CTFs and lab work keep your skills sharp.*
- ***Keep learning deliberately:*** *the later courses, and new topics as they emerge, like AI security.*

And, threaded through all of it, ***stay safe and ethical*** *(lesson 9). As your skills grow, so does your power to cause harm, and the temptation to "just try" something you are not authorized to touch. The professionals who have long, respected careers are the ones who never cross that line. Your reputation for integrity is the most valuable thing you will build, and the easiest to destroy. Guard it from day one.*

8. Your Next Step

You have finished Security Fundamentals. You now understand what security is, how attacks and defenses work, how to think about risk, the rules that keep your skills legal, the frameworks that organize the work, and the roles you could grow into. That is a real foundation, and more than many working people in the field started with.

Your next step is simple and concrete:

1. ***Begin the next Foundation course****, How Computers Work, and continue the Foundations in order. Do not skip ahead, however tempting.*
2. ***Plan to build your home lab*** *as soon as that course arrives, so you have somewhere to practise.*
3. ***Start a write-up habit:*** *keep notes on everything you learn, from this lesson onward.*
4. ***Decide, loosely, which path calls to you*** *(offensive, defensive, engineering), knowing you can change your mind.*

Security is a field where curiosity, persistence, and integrity matter more than raw talent, and where demand far exceeds supply. If you keep going, practise hands-on, build a portfolio, and never cross the ethical line, you can go a very long way. You have taken the first step. Keep going, and welcome to the field.

Common Mistakes

1. Rushing or skipping the Foundations. *Almost everyone who struggles later skimmed the basics. Build them solidly and in order; they are what separate real understanding from recipe-following.*

2. Learning by reading alone. *You cannot learn security without doing it. Practise everything hands-on in your home lab.*

3. Waiting for a job to start a portfolio. *Build evidence now, write-ups, projects, documented lab work. It proves ability before you have experience.*

4. Thinking you must master everything before specializing. *Do the Foundations, then pick the path that excites you. You can broaden later.*

5. Practising on systems you do not own. *Use your home lab and legal practice platforms. The ethics rule does not pause while you learn.*

6. Treating learning as a one-time event. *Security changes constantly. Staying current and staying ethical are career-long habits.*

Summary

Idea	Remember
Program order	Foundations → a path (Offensive/Defensive/Engineering) → Specialized
Foundations first	Do them in order, don't rush; they are the real base
Choosing a path	Pick what excites you; if unsure, start Defensive; broaden later
Home lab	Your safe, legal place to practise; do, don't just watch
CTFs and challenges	Free, legal, graded practice that compounds into skill
Portfolio	Write-ups and projects prove ability and get you hired; keep it honest
Stay current and safe	Keep learning; never cross the ethical line

Check Your Understanding

1. What are the five stages of the program, in order?
2. Why must the Foundations come first, and what is the risk of rushing them?
3. If you are unsure which path to choose, which is suggested, and why?
4. Why can't you learn security by reading alone, and what is the main place to practise?
5. What makes CTFs and practice platforms a legal and effective way to practise?
6. Name two good portfolio pieces, and say why write-ups are especially valued.
7. Why does staying ethical get harder, and more important, as your skills grow?

Answers

1. *Foundations, Offensive, Defensive, Engineering, and Specialized (including careers and the capstone).*
2. *Because each stage depends on the ones before it; you can't attack or defend what you don't understand. Rushing them is the most common cause of struggling later.*
3. *Defensive, because it has the broadest job market, teaches you to think about attacks from the receiving end, and gives a strong base to pivot from.*
4. *Because skill comes from doing, not just knowing, like a locksmith who must practise, not only read. The main place to practise is your own home lab.*
5. *The targets are built deliberately for practice, so using them is legal; the challenges are graded so you know when you succeed; and they build real problem-solving skill.*
6. *Any two: write-ups, a blog/notes site, security tools/scripts, documented lab work, responsible bug-bounty results. Write-ups are prized because they show both technical skill and clear communication.*
7. *Because greater skill means greater power to cause harm and more temptation to test things you're not authorized to touch; the professionals with long careers never cross that line, and integrity is the most valuable and most fragile thing you build.*

Practice

This final practice is a plan for yourself.

1. *Write your **learning schedule**: realistically, how many hours a week can you give this, and when? (Consistency beats intensity: a steady few hours a week compounds.)*
2. *Write down the **path** you are leaning toward (offensive, defensive, or engineering) and one sentence on why. It is fine to change later.*
3. *Commit to a **write-up habit**: decide where you will keep your notes (a document, a blog, a notebook), and write a short one for this lesson, what were the three most useful things you learned in Security Fundamentals, and why.*

A Worked Example

1. *Schedule: about six hours a week, one hour on weekday evenings and a longer session on Saturday morning.*
2. *Leaning Defensive, because I like investigating and noticing patterns, and it has the broadest job market to start in.*

3. *Write-ups: a simple blog. First entry, three most useful things from this course: the three questions (read/change/stop), because they frame every problem; "never trust the client / least privilege", because it reshapes how I see systems; and the one ethics rule, because it's the line my whole future career depends on.*

References

These are the main sources used to write this book, and good places to learn more. Security guidance and figures change over time, and laws differ by country, so check the current version of any source before relying on a specific detail. Web addresses were correct at the time of writing.

Frameworks and standards

NIST Cybersecurity Framework 2.0 (2024) — the six functions used in section 10. https://www.nist.gov/cyberframework

NIST SP 800-30, Guide for Conducting Risk Assessments — the risk ideas in section 3. https://csrc.nist.gov/pubs/sp/800/30/r1/final

NIST SP 800-63, Digital Identity Guidelines — the password and authentication guidance in section 6. https://pages.nist.gov/800-63-3/

CIS Critical Security Controls v8.1 — the prioritized controls and Implementation Groups in section 10. https://www.cisecurity.org/controls

ISO/IEC 27001 and 27002 — the international standard and its control catalogue, section 10. https://www.iso.org/standard/27001

MITRE ATT&CK — the knowledge base of attacker behavior, sections 5 and 10. https://attack.mitre.org

Attacks, risks, and guidance

OWASP (Open Worldwide Application Security Project) — the Top 10 and much more, sections 5 and 10. https://owasp.org

Verizon Data Breach Investigations Report (DBIR) — evidence on how attacks actually start, sections 4 and 5. https://www.verizon.com/business/resources/reports/dbir/

CISA (US Cybersecurity and Infrastructure Security Agency) — practical defensive guidance and advisories. https://www.cisa.gov

Have I Been Pwned — checking whether your accounts appear in known breaches, section 6. https://haveibeenpwned.com

Law and ethics

Computer-crime and data-protection law differs by country and changes. ***Confirm the current law in Cambodia*** *before relying on any legal detail in section 9. The following are widely documented references for the shape of such law:*

(US) Computer Fraud and Abuse Act — https://www.justice.gov/criminal/ccips

(EU) General Data Protection Regulation (GDPR) — https://gdpr.eu

Careers

NIST NICE Workforce Framework for Cybersecurity *— the roles described in section 11. https://niccs.cisa.gov/workforce-development/nice-framework*

Every figure and legal statement in this book should be re-checked against these sources before publication, and dated.